

Web Information Extraction and Retrieval

Marko Bajec

Laboratory for Data Technologies, UL FRI

About the course

- Lecturers
- Laboratory work
- Seminars
- Grading system
- Literature

Course content

- Module 1: Web crawling
- Module 2: Structured data extraction
- Module 3: Information Retrieval and Web Search

Module 1: Web crawling

Module content

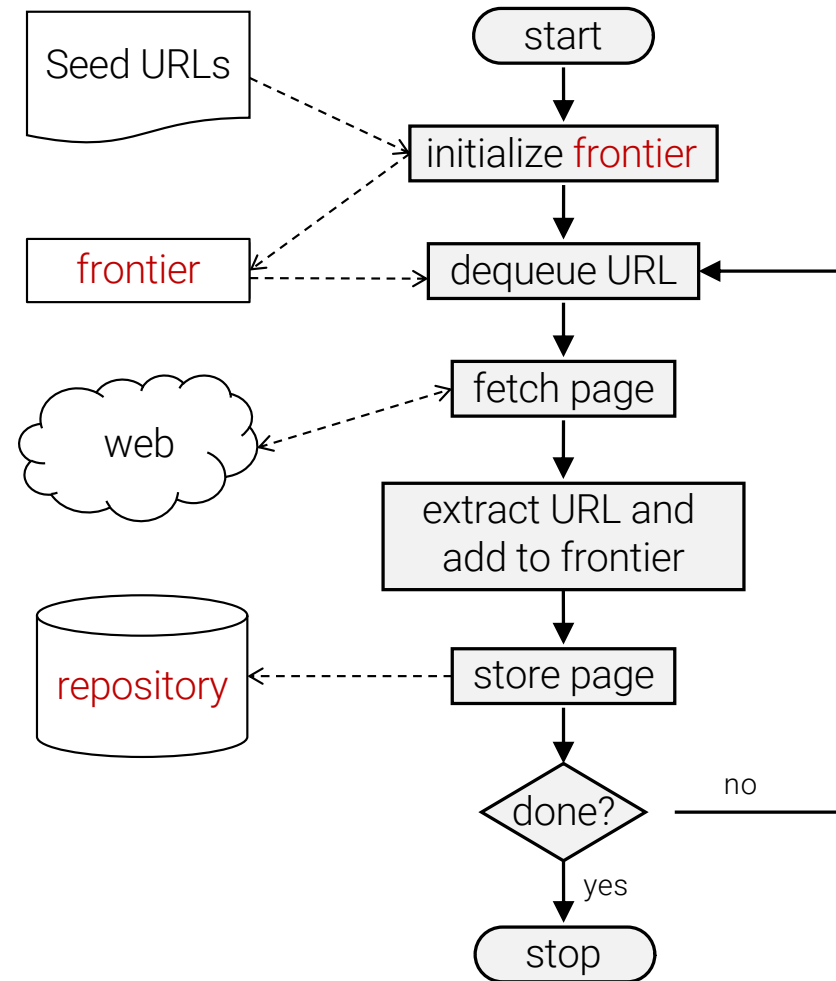
- Architecture
 - Basic
 - Parallel
 - Scalable
- Types
 - Universal
 - Focused
 - Topic-based
- Ethics

Web crawlers

- Description
- Challenges
- Application
- Abuse

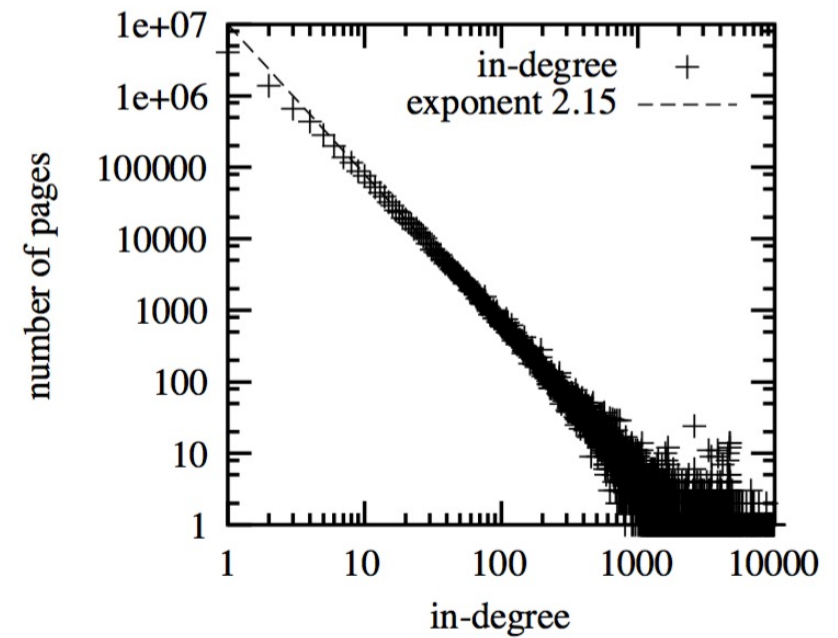
Basic architecture

- Frontier
- Seed pages
- Repository
- Implementation decisions:
 - Storage for URLs
 - URL selection strategy
 - Strategy if Frontier full
- Challenges and limitations



Breadth-first crawlers

- Frontier implementation (FIFO)
- Size limitations of frontier
- Is breadth-first crawling **random**?



Crawling history

- Reasons
- Implementation (disk, memory, hash tables)
- Duplicate detection (on Frontier and Repository level)
- Time complexity

Preferential crawlers

- Preferential crawling – **Best-first** crawler
- Preference set based on web topology and page content
 $f: URL \rightarrow \mathcal{R}$
- Selection of seed pages important
- Implementation:
 - Frontier - sorted dynamic list $O(\log_2 F)$
 - Lookup table: hash table
- What if the Frontier is full?

Implementation details

- Fetching
- Parsing
- Stopwords removal and stemming
- Link extraction and canonicalization
- Spider Traps
- Page repository
- Concurrency

Fetching₁

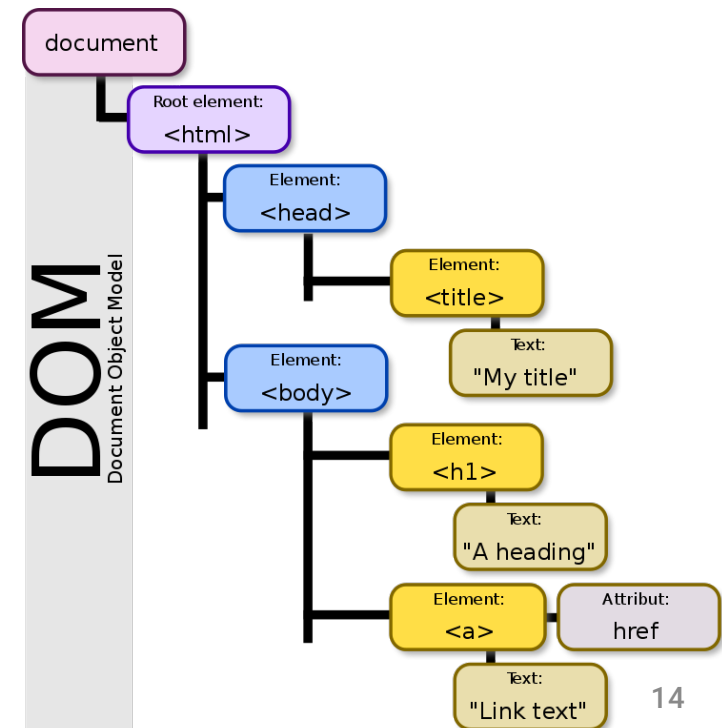
- Implementation (web client, HTTP requests)
- HTTP request – GET, POST, DELETE, PATCH, HEAD
- HTTP response codes:
 - 1xx (Informational) → Provides protocol-level information.
 - 2xx (Success) → The request was successfully processed.
 - 3xx (Redirection) → Further action is needed to complete the request.
 - 4xx (Client Error) → Indicates an issue on the client side.
 - 5xx (Server Error) → Indicates an issue on the server side.

Fetching₂

- Limiting response time and page size
- HTTP headers (useful information...)
 - Accept-Ranges → Indicates whether the server supports partial requests
 - Expires → Specifies when the response becomes stale.
 - Last-Modified → Shows the last modification date of the resource
 - ETag → Provides a unique identifier for a specific version of a resource
- Error-checking and exception handling
- Keeping statistics
- Program language support

Parsing

- Parsing HTTP payload (link extraction, content analysis,...)
- Document Object Model (DOM)
- Cleaning HTML code prior parsing
- Basic format: HTML
- Other formats:
 - PDF, plain text, MS Power Point, MS Word ...
 - Flash, SVG, AJAX, XML-based ...



Stopword removal and stemming

- Stopwords (common words)
 - articles, conjunctions ...
- Stemming (removing inflated parts of words)
 - run, runs, running → run
 - argue, argues, arguing → argu
- Stemming ≠ lemmatization

Link extraction and canonicalization

- HTML link representation

`<a href="http://datalab.si/">Laboratory for Data Technologies</a>`

- Filtering

- Content type, HTTP head request
- Dynamic pages

- Autonomous querying (hidden web)

- URLs need to be absolute

- Canonicalization

Description and transformation	Example and canonical form
Default port number Remove	http://cs.indiana.edu:80/ http://cs.indiana.edu/
Root directory Add trailing slash	http://cs.indiana.edu http://cs.indiana.edu/
Guessed directory* Add trailing slash	http://cs.indiana.edu/People http://cs.indiana.edu/People/
Fragment Remove	http://cs.indiana.edu/faq.html#3 http://cs.indiana.edu/faq.html
Current or parent directory Resolve path	http://cs.indiana.edu/a/./b/ http://cs.indiana.edu/b/
Default filename* Remove	http://cs.indiana.edu/index.html http://cs.indiana.edu/
Needlessly encoded characters Decode	http://cs.indiana.edu/%7Efil/ http://cs.indiana.edu/~fil/
Disallowed characters Encode	http://cs.indiana.edu/My File.htm http://cs.indiana.edu/My%20File.htm
Mixed/upper-case host names Lower-case	http://CS.INDIANA.EDU/People/ http://cs.indiana.edu/People/

Spider traps

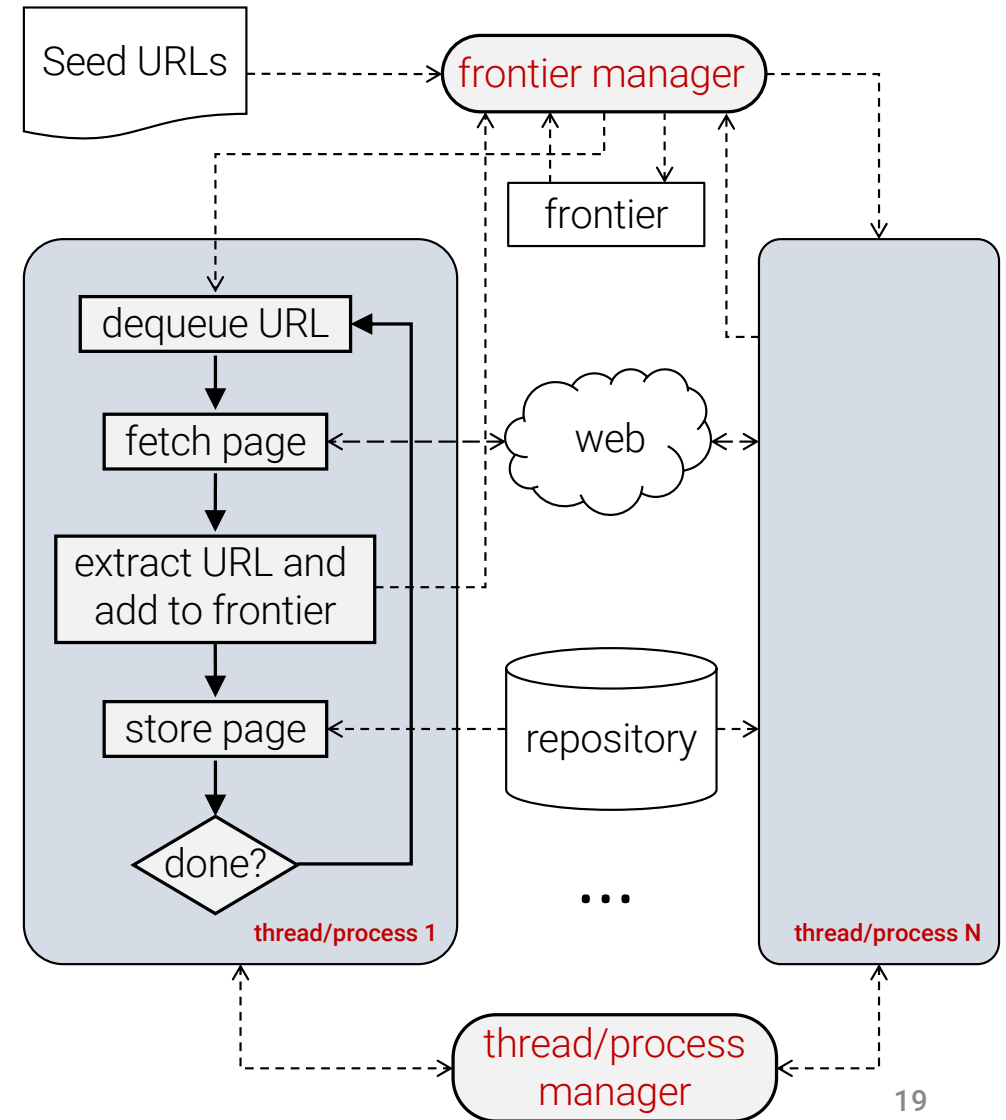
- Changing URLs of dynamically created links
- Same pages with new URLs
- Negative side effects
 - for crawlers (wasted bandwidth and disk space)
 - for servers (unintended DOS)
- Avoiding spider traps
 - refusing sending/receiving cookies,
 - limiting URL length
 - limiting number of pages from a given domain

Page repository

- File system
 - each page in a separate file
 - many pages in one file (plus hash table)
- Databases
 - embedded, e.g. Berkeley DB (key value store)

Concurrency

- Problematical resources
 - network,
 - CPU,
 - disk
- Improvements of sequential crawler
 - multithreading
 - multiprocessing
- Dealing with empty Frontier



Universal crawlers

- Performance
 - Hundreds of thousands of pages fetched and indexed per second
 - Scalability
- Policy
 - Balancing coverage, importance, freshness

Coverage vs. freshness vs. importance...

- Indexing entire internet
- Importance measures (in-degree, PageRank, ...)
- Do nothing approach
- Fetching statistically representative sample of Web

$$\Pr(\textit{accept}(p) | \textit{crawl}(p)) * \Pr(\textit{crawl}(p))$$

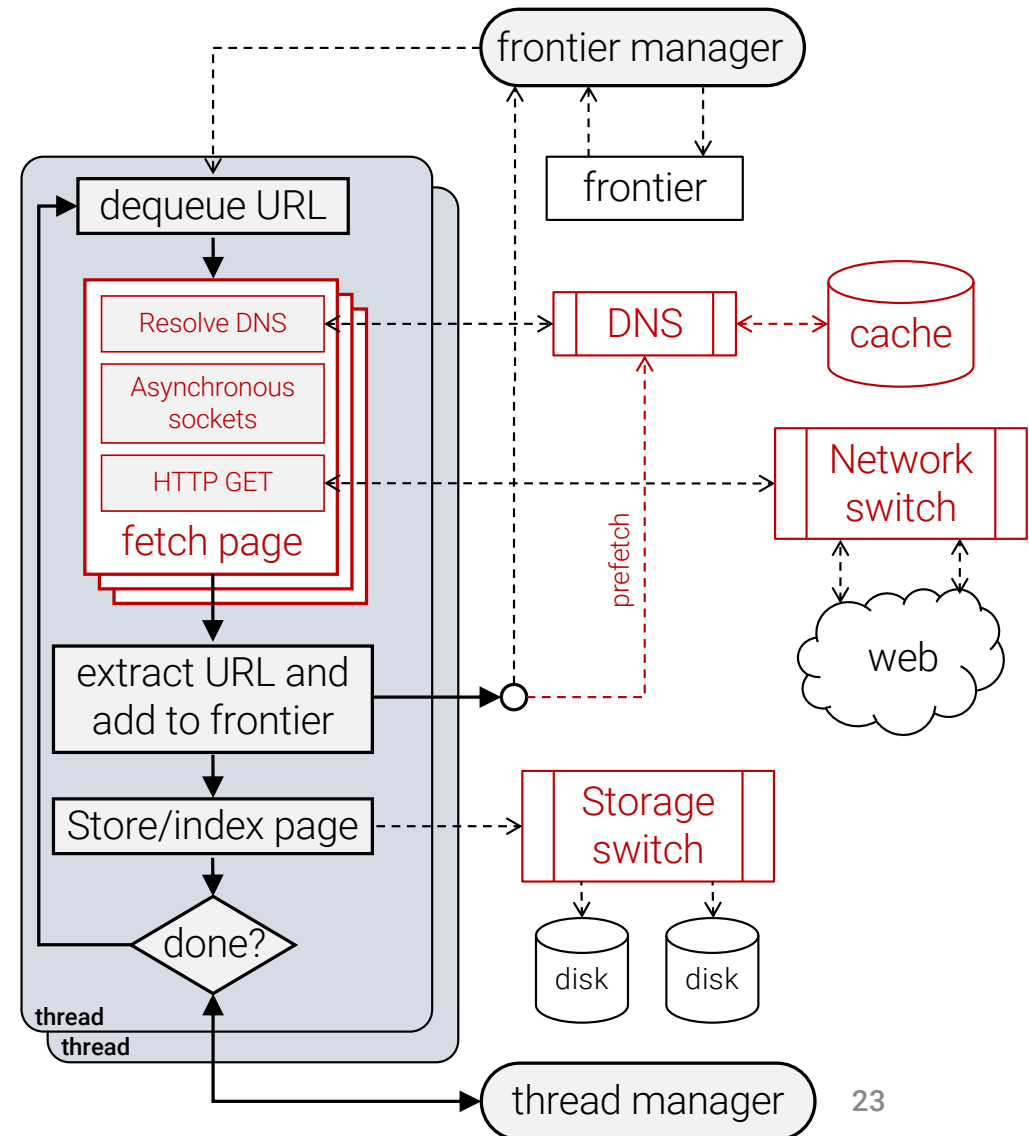
Coverage vs. freshness vs. importance*

- Coverage vs. freshness
- Dynamics of the Web (content, link structure)
 - 8% new pages every week
 - 62% of content on new pages is new
 - 25% new links per week
 - Once published, pages rarely change
 - Degree of change better predictor of future change than frequency of change

*Ntoulas, A., J. Cho, and C. Olston. *What's new on the web?: the evolution of the web from a search engine perspective*. In Proc. of International Conference on World Wide Web (WWW-2004), 2004.

Scalability

- Asynchronous sockets
- Multiple URL queues
- Working with DNS
 - Connections over UDP
 - employing fast in-memory cache
 - prefetching URLs
- Multiple network connections
- Use of SAN

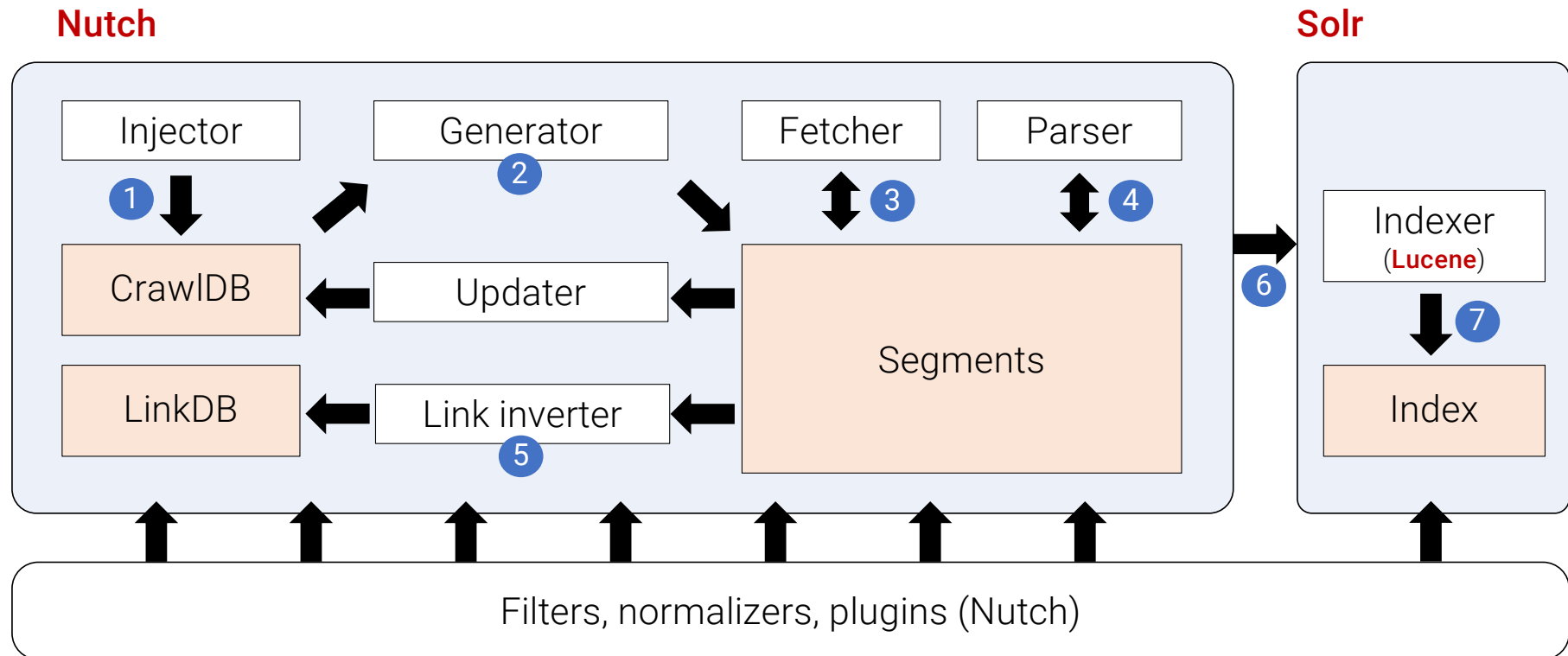


Apache Nutch

- Main characteristics
 - Extendable
 - Can work on distributed architectures
 - Written in Java
 - Independent data structures
- Components
 - Compose of five components
 - Crawler, Searcher, Indexer, Parser, Storage
- Usage
- Branches
 - Two main forks: 1.x in 2.x (Apache Gora)

<http://nutch.apache.org/>

Apache Nutch architecture



Focused crawlers

- Crawling pages of interest
- Taxonomy tree



Web Directory of High-quality Resources



Arts

[Movies](#), [Television](#), [Music](#)...



Games

[Video Games](#), [RPGs](#), [Gambling](#)...



Kids and Teens

[Arts](#), [School Time](#), [Teen Life](#)...



Reference

[Maps](#), [Education](#), [Libraries](#)...



Shopping

[Clothing](#), [Food](#), [Gifts](#)...



Business

[Jobs](#), [Real Estate](#), [Investing](#)...



Health

[Fitness](#), [Medicine](#), [Alternative](#)...



News

[Media](#), [Newspapers](#), [Weather](#)...



Regional

[US](#), [Canada](#), [UK](#), [Europe](#)...



Society

[People](#), [Religion](#), [Issues](#)...



Computers

[Internet](#), [Software](#), [Hardware](#)...



Home

[Family](#), [Consumers](#), [Cooking](#)...



Recreation

[Travel](#), [Food](#), [Outdoors](#), [Humor](#)...



Science

[Biology](#), [Psychology](#), [Physics](#)...



Sports

[Baseball](#), [Basketball](#), [Soccer](#)...



World

[Deutsch](#), [Français](#), [日本語](#), [Italiano](#), [Español](#), [Русский](#), [Nederlands](#), [Polski](#), [Türkçe](#), [Dansk](#), [简体中文](#)...

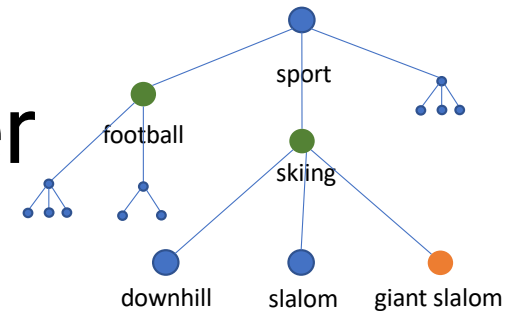
<http://odp.org/>

656

Like

Share

Classifier based focused crawler



- Classification
 - Each crawled page is assigned **relevance score**: $R(p) = \sum_{c \in c^*} \Pr(c|p)$
 - where c^* denotes categories of interest
- Soft focused strategy
 - Links of page p are given preference $R(p)$ when written to the Frontier.
- Hard focused strategy
 - Find leaf category c_{leaf} that most likely include p
 - If c_{leaf} or any of its ancestors in focused categories $\rightarrow$ add links(p) to Frontier (using $R(p)$).

Text classification with naïve Bayes

$$\Pr(c|d) = \frac{\Pr(d|c) * \Pr(c)}{\Pr(d)}$$

where $c \in \mathcal{C}$ is a category from a set of categories $\mathcal{C}$ and d represents the crawled page. If d represented using multinomial document model then:

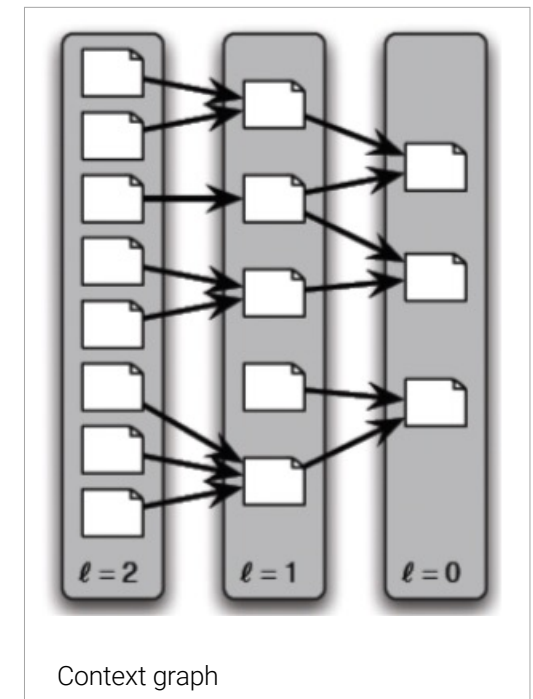
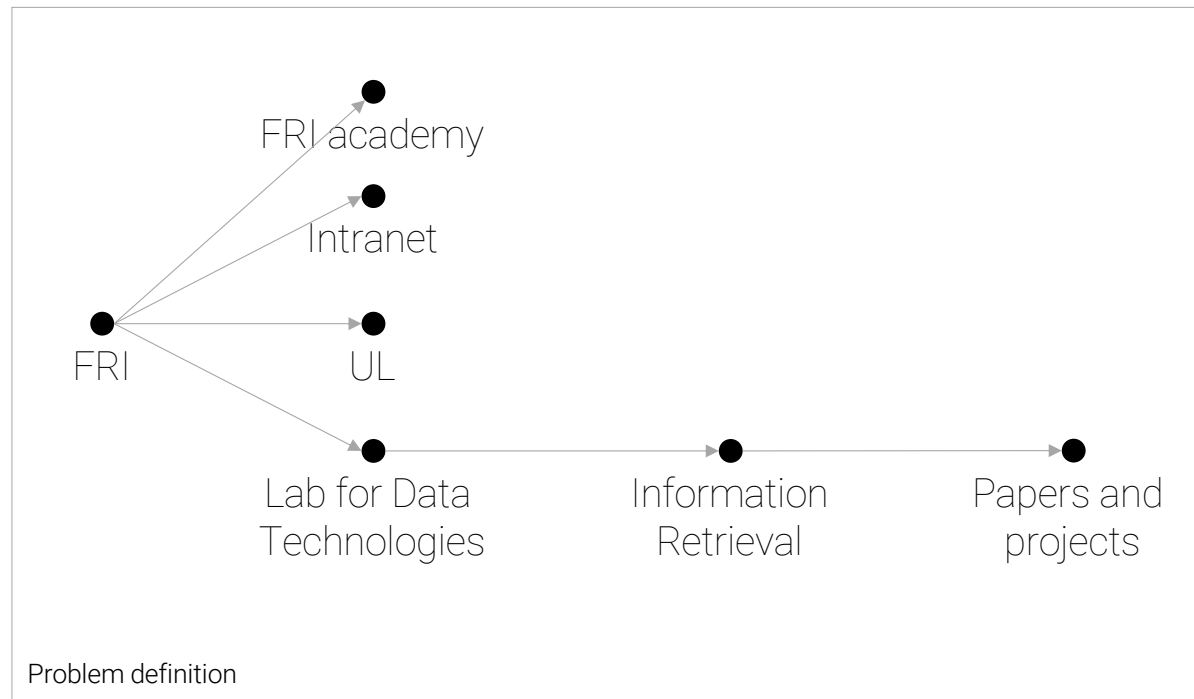
$\Pr(c|d) \approx \Pr(c) \prod_{i=1}^{|V|} \Pr(w_i|c)^{f_{w_i}}$, where V is the vocabulary of distinct words w_i in document d that we would like to classify and f_{w_i} is the frequency of words w_i in d . We are looking for $c_j \in \mathcal{C}$ where

$$c_j = \underset{c_k \in \mathcal{C}}{\operatorname{argmax}} \Pr(c_k) \prod_{i=1}^{|V|} \Pr(w_i|c_k)^{f_{w_i}}$$

Exercise

- Taxonomy: c_1 =sport, c_2 =politics
- Training dataset:
 - “A great game” → sports
 - “The election was over” → politics
 - “Very clean match” → sports
 - “A clean but forgettable game” → sports
 - “It was a close election” → politics
- Is the following text about sports or politics?
 - “A very close game” → ?

Context-based focused crawler ...



Context-based focused crawler*

■ Procedure:

- Use TFIDF to build vocabulary V of important words w for the topic of interest.
- Build Naïve Bayes classifiers for each layer: $\Pr(l_x|p) = \Pr(w_1|l_x) * \Pr(w_2|l_x) * \dots * \Pr(w_n|l_x) * \Pr(l_x)$, where $w_{1\dots n} \in (p \cap V)$
- If probability of layer with $\max \Pr(l|p) < threshold \rightarrow$ classify page p to *others* otherwise put to queue of the corresponding layer.
- Link to be followed next is selected from a nonempty queue of the lowest layer number.

*Diligenti, M., F. Coetzee, S. Lawrence, C. Giles, and M. Gori. *Focused crawling using context graphs*. In Proceedings of International Conference on Very Large Data Bases (VLDB-2000), 2000.

Tunneling...

- Limitations of Best-first approach
- Tunneling
 - Which links from irrelevant pages to follow?
 - How long to follow a tunnel?
 - How to estimate the potential of a certain tunnel?

Tunneling...

■ Concepts*:

- **Nuggets** and **duds**: relevant and irrelevant pages
- **Path**: a sequence from one nugget to another one
- **Threshold**: minimal similarity with topics to consider p as a nugget
- **Cutoff**: max length of a path
- **Distance**: length from the last nugget

$$distance = \begin{cases} 0, & \text{if } p \text{ nugget} \\ 1 + distance(parent(p)), & \text{otherwise} \end{cases}$$

*Bergmark, D., Lagoze, C., & Sbityakov, A. (2002), *Focused Crawls, Tunneling, and Digital Libraries*, In proc. of the European Conference on Digital Libraries (ECDL) 2002, Rome, September 2002, 91-106.

Tunneling

- Experimental findings (on 500.000 documents)
 - Most common distances are 7 and 8
 - Better parents have better children, but the opposite is not true.
 - For high-scoring parents, highly scoring grandparents important.

Distance incorporating path history

$$distance = \begin{cases} 0, & \text{if } p \text{ nugget} \\ \min(1, (1 - c)e^{\frac{2d_p}{c}}), & \text{otherwise} \end{cases}$$

where d_p : parent distance, c : current node correlation, C : cutoff

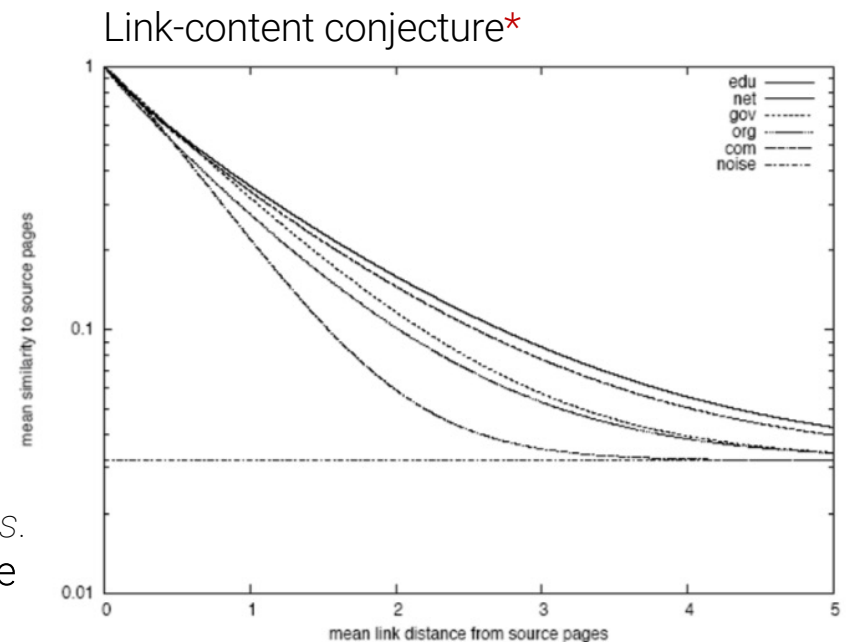
Topical crawlers

- Limited prior knowledge → real time crawling (no index), calculating relevance on-the-fly
- Pros:
 - fresh hits (access to documents not yet indexed)
- Cons:
 - slower than index-based search,
 - less accurate ranking (no global measures)
- Cues to explore and other influencing parameters

Cues from words and hyperlinks*

- Lexical metrics
- Link metrics:
 - distance*
 - co-citation (semantic similarity)
 - Hubs (importance)
 - bibliographic coupling (semantic s.)

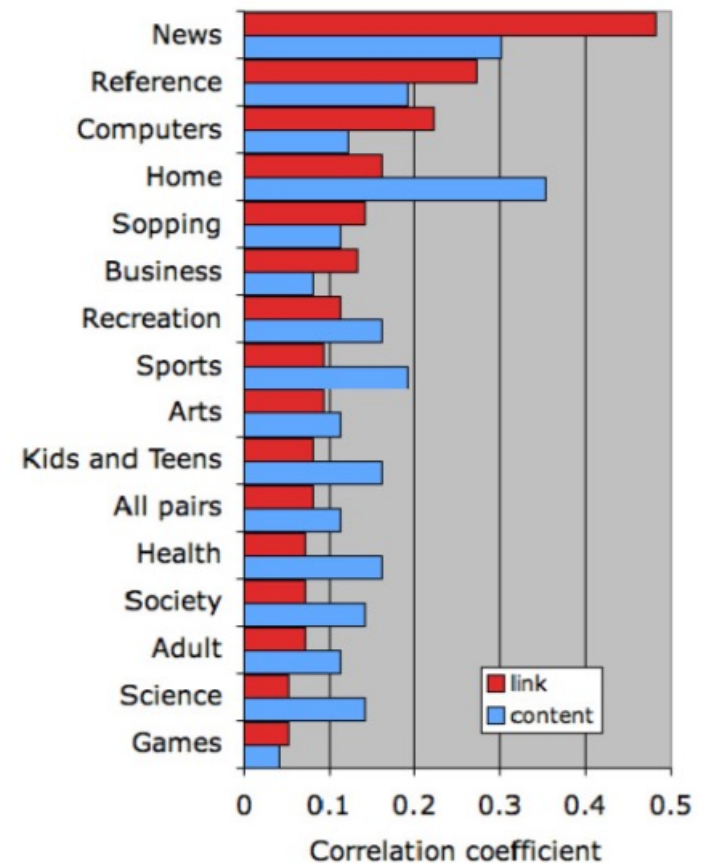
*Menczer, F. *Lexical and semantic clustering by web links*. Journal of the American Society for Information Science and Technology, 2004, 55(14): p. 1261-1269.



Link content and cluster conjecture*

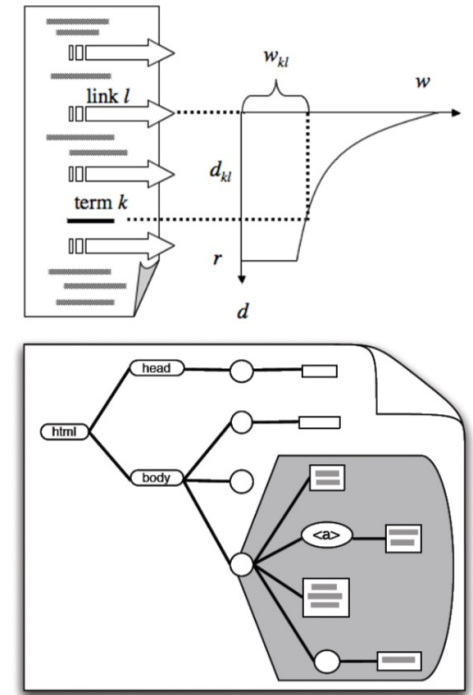
- Correlation between similarity measures:
 - content-based,
 - link-based (bag of links), and
 - semantic-based (ODP – ground truth).

*Menczer, F. *Mapping the semantics of web text and links*.
Internet Computing, IEEE, 2005, 9(3): p. 27-36.



Variations of best-first crawlers

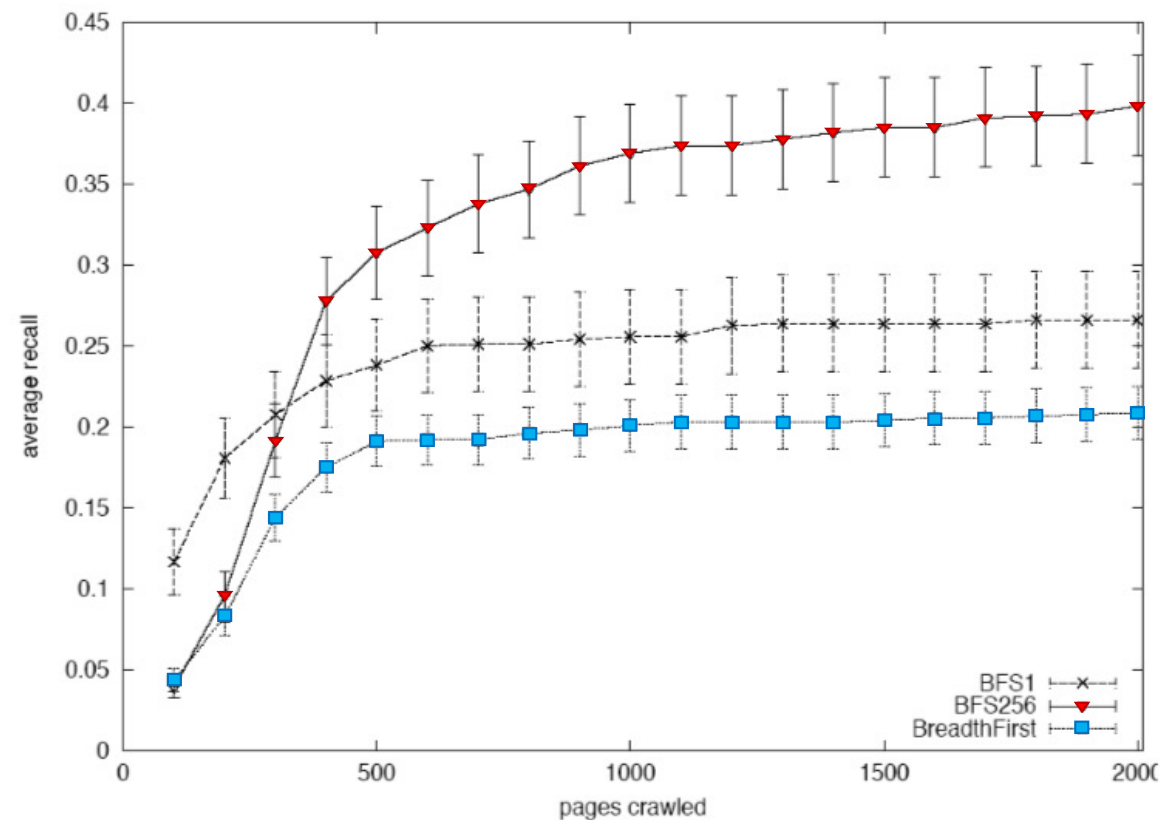
- Naïve best-first crawlers
- InfoSpiders [1]
- Clever crawler [2]
- SharkSearch [3]



1. Menczer, F. and R. Belew. *Adaptive retrieval agents: Internalizing local context and scaling up to the Web*. **Machine Learning**, 2000, 39(2): p. 203- 242..
2. Chakrabarti, S., B. Dom, S. Kumar, P. Raghavan, S. Rajagopalan, A. Tomkins, D. Gibson, and J. Kleinberg. *Mining the Web's link structure*. **Computer**, 2002, 32(8): p. 60-67.
3. Hersovici, M., M. Jacovi, Y. Maarek, D. Pelleg, M. Shtalhaim, and S. Ur. *The shark-search algorithm. An application: tailored Web site mapping*. **Computer Networks**, 1998, 30(1-7): p. 317-326.

Best-N-first crawler

- Visiting of less promising pages to avoid “local optimums”
- Best-N-first crawler
 - follow best N leads
 - fetch all, merge-sort, repeat
 - Exploitation vs exploration



Adaptive topical crawlers

- Static vs adaptive strategy (incremental learning)
- Example of adaptive strategy:

```
select features  $fe_i \in F$ 
while not enough relevant pages crawled do
  crawl new page  $p$ 
  calculate page priority  $P(p) = f(\lambda(fe_1), \lambda(fe_2), \dots, \lambda(fe_m))$ 
  on every  $n$  crawled pages
    recalculate interest factors  $\lambda(fe_i)$ 
  reorder frontier
end
```

Interest factor

$\lambda(fe_m)$

Intelligent crawler*

■ Features

- Content based $\rightarrow I_c(c)$
- URL token based $\rightarrow I_u(c)$
- Link based $\rightarrow I_i(c)$
- Sibling based $\rightarrow I_s(c)$

$$PriorityValue = w_c \log(I_c(c)) + w_u \log(I_u(c)) + w_i \log(I_i(c)) + w_s \log(I_s(c))$$

*Aggarwal, C., F. Al-Garawi, and P. Yu. *Intelligent crawling on the World Wide Web with arbitrary predicates*. In Proceedings of 10th International Conference on World Wide Web (WWW-2001), 2001.

Reinforcement learning (RL)

- Reinforcement learning (RL) is technique to find optimal solution by trial and error.
- RL imitates reward-motivated behavior.
- Simulation of real-life learning
 - try different strategies
 - learn from the feedback
 - act according to the best strategy



RL video demonstration



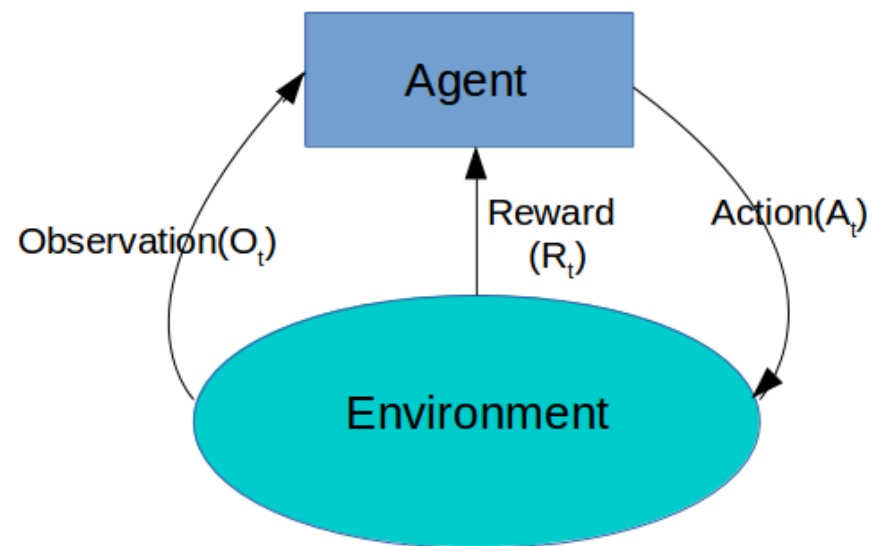
DRL

Deep Reinforcement Learning

Formalization

- Key **concepts** in RL:

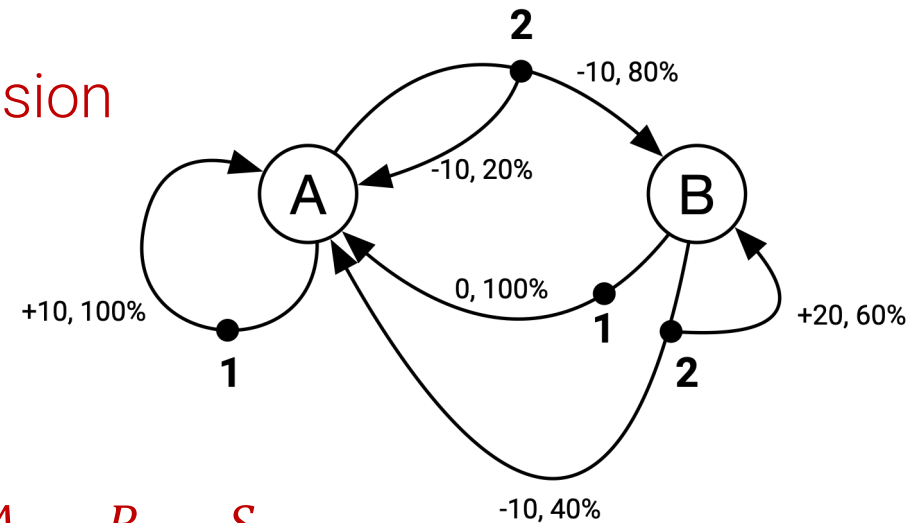
- agent,
- state,
- action,
- environment,
- reward.



RL – modeling environment

■ Environment: A finite Markov Decision Process (MDP)

- Discrete time $t = 1, 2, 3$
- A finite set of states
- A finite set of actions
- A finite set of rewards
- Life is trajectory $\dots S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, S_{t+2}, \dots$ with Markov dynamics:



$$p(r, s' | s, a) = \text{Prob}[R_{t+1} = r, S_{t+1} = s' | S_t = s, A_t = a]$$

RL – policies

- Policy π – defines for each state s what action a to take
- Deterministic policy: $a = \pi(s)$
- An agent following a policy: $A_t = \pi(S_t)$
- The agent's goal is to choose actions A_t in a way to maximize the discounted sum of future rewards:

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

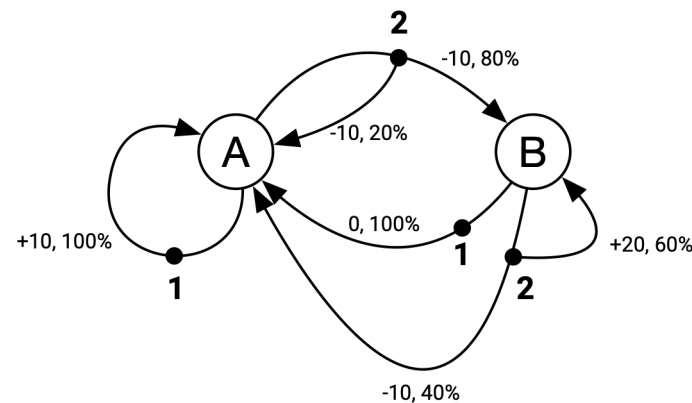
RL – Action-value functions

- An **action-value function** tells how good is to be in a state, take an action and then follow a policy:

$$q_{\pi}(s, a) \approx \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a \right]$$

Action-value function

State	Action	Value
A	1	130,39
A	2	133,77
B	1	166,23
B	2	146,23



RL – State value function

- A **state value function** tells, how good it is for the agent to be in a certain state.
- The value of a state **s** under a policy **π** , denoted **$v_\pi(s)$** , is the expected return when starting in **s** and following **π** thereafter.
- For MDPs, we can define **v_π** as:

$$v_\pi(s) \approx \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right]$$

RL – Optimal policies...

- An optimal policy π_* is **better** than other policies. It maximizes the state and action-value functions:

$$q_{\pi_*}(s, a) = \max_{\pi} q_{\pi}(s, a), \quad v_{\pi_*}(s) = \max_{\pi} v_{\pi}(s)$$

- Bellman optimality equations:

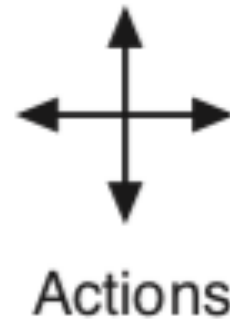
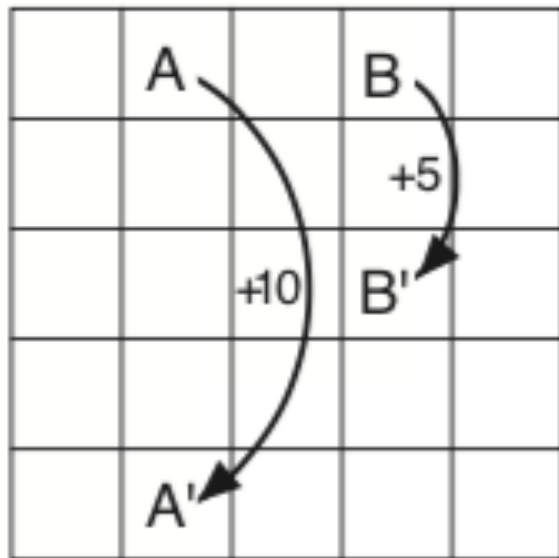
$$v_{\pi_*}(s) = \max_a q_{\pi_*}(s, a) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi_*}(s')],$$

$$q_{\pi_*}(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_{\pi_*}(s', a') \right]$$

RL – Optimal policies

- Given the optimal value functions, it is easy to act optimally:
 - If we have v_{π^*} then we just need to check which states that are reachable from the current state gives the best value.
 - If we have q_{π^*} then we just select the best action.
- Optimal policy is **greedy** with respect to the optimal value functions.

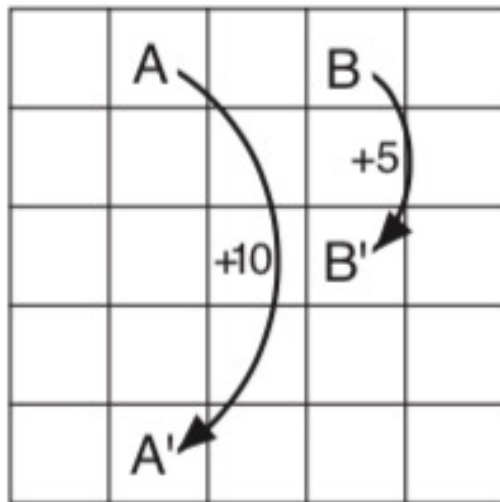
Gridworld example...



3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

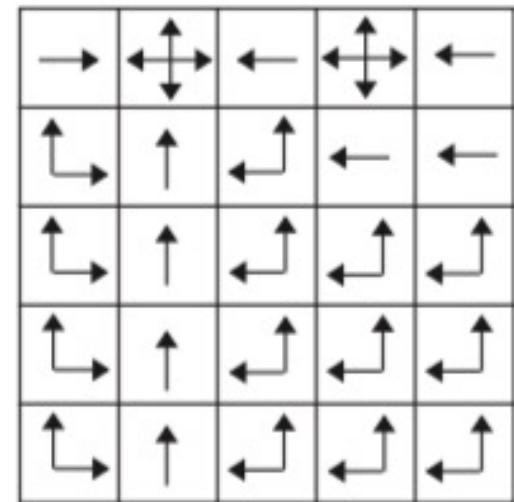
Random action policy

Gridworld example



22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

Optimal state-value function



Optimal policy

Q-learning

- The simplest RL algorithm:

- Initialize an array $Q(s, a)$
- Choose actions in any way (e.g. to take all states several times)
- On each time step, change one element of the array:

$$\Delta Q(S_t, A_t) = \alpha \left(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right)$$

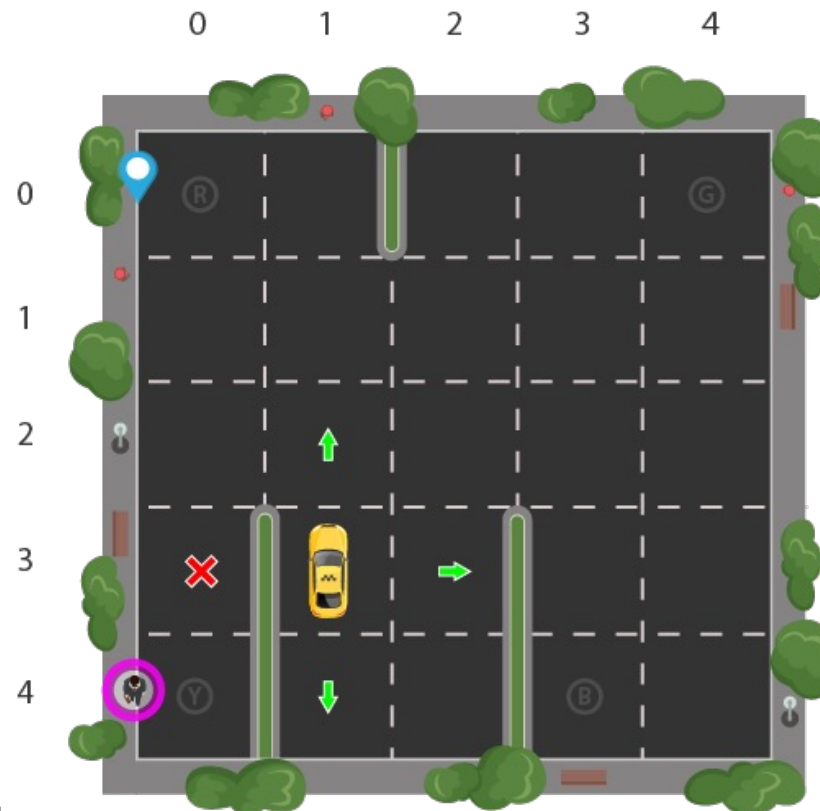
- If desired, reduce step-size parameter α over time.

- Theorem*: for appropriate choice of α , Q converges to q_{π^*} and its greedy policy to an optimal policy π_* (Watkins & Dayan, 1992).

*Watkins, C. J. C. H., Dayan, P. (1992). *Q-learning*. Machine Learning, 8:279–292.

Example of using Q-learning

- SmartCab



Reinforcement learning vs ML

- Specifics:
 - no supervisor,
 - importance of time,
 - delayed rewards,
 - agent action effects its next input.
- History abstraction (Markov state)
- Algorithms (Q-learning, SARSA, DQN, DDPG)

RL-based Focused Crawler*

- Goal of focused crawler
 - Collect relevant pages while avoid irrelevant (due to resources)
 - Selection of most promissing link may not lead to relevant pages in a long run.
- RL agent evaluates if a link selection can yield a long-term optimal reward...

*Han, M., Wuillemin, P., Senellart, P. *Focused Crawling Through Reinforcement Learning*, Lecture Notes in Computer Science, Springer International Publishing AG, T. Mikkonen et al. (Eds.): ICWE 2018, LNCS 10845, pp. 261–278, 2018.

Problem modelling

- Representation
 - States – web pages
 - Actions – hyperlinks

- Search space simplification:
 - Pages with same feature values represented as a single page
 - Links with same feature values represented as one link.

Features of pages and links

■ Pages

- Relevance to target topic and selected categories
- Relevance change
- Average relevance of parent pages
- Distance from the last relevant page

■ Links

- Relevance of target topic and selected categories,
- Average relevance of parents

MDPs with Prioritizing updates

- Using prioritized URLs, focused crawler would select best link to go – no matter of its current position!
- Crawling agent has to follow this principle.

- Prioritized updates

to update state s with change δ in $U(s)$:

update $U(s)$

priority of $s \leftarrow 0$

for each predecessor s' of s :

priority $s' \leftarrow \max$ of current and $\max_a \delta \hat{T}(s', as')$

Linear Function Approximation

- In **large state-action spaces**, a tabular representation of action-value functions impossible.
- Action values are represented with **linear function approximation**:

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s, a) = \sum_{i=1}^d w_i x_i(s, a)$$

where $\mathbf{x}(s, a)$ is a feature vector and $\mathbf{w}$ weight vector.

Gradient descent update on SARSA

- At each time step, weight vector $\mathbf{w}$ is updated.
- Gradient descent method is used (in direction that minimizes TD error):

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}_t) - \hat{q}(s_t, a_t, \mathbf{w}_t)] \nabla \hat{q}(s_t, a_t, \mathbf{w}_t)$$

Partial derivatives of the function

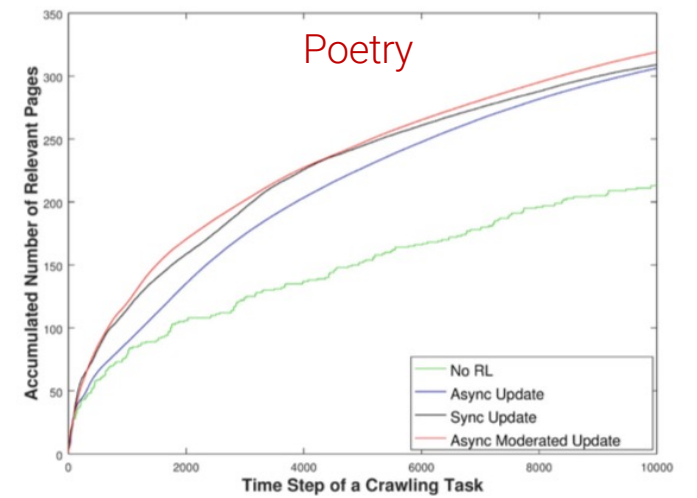
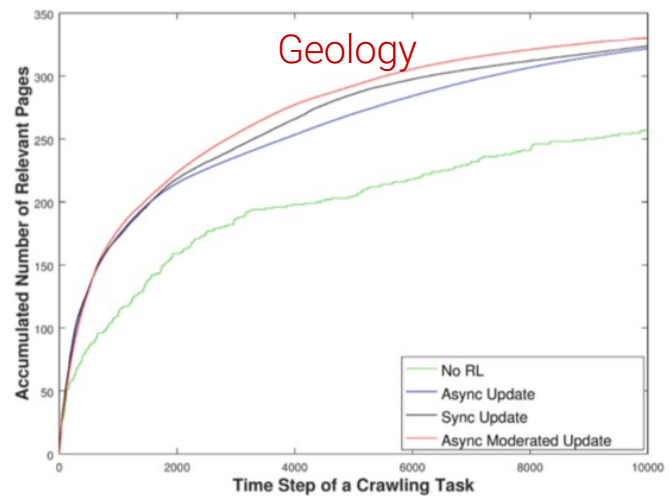
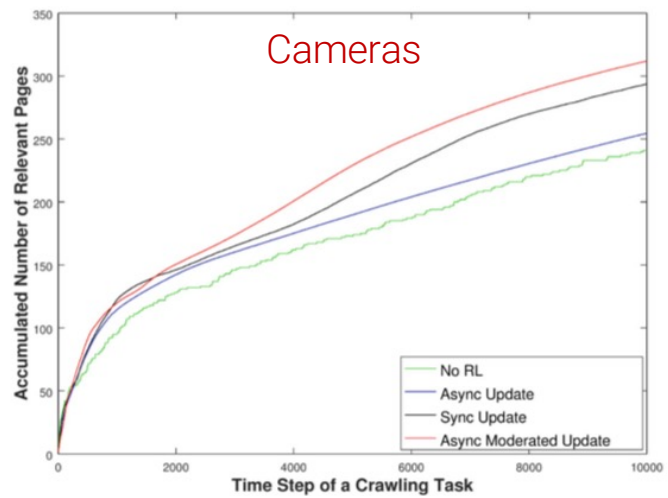
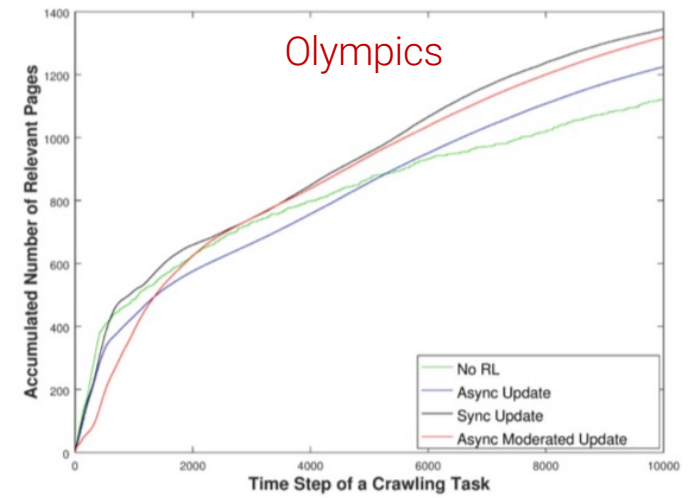
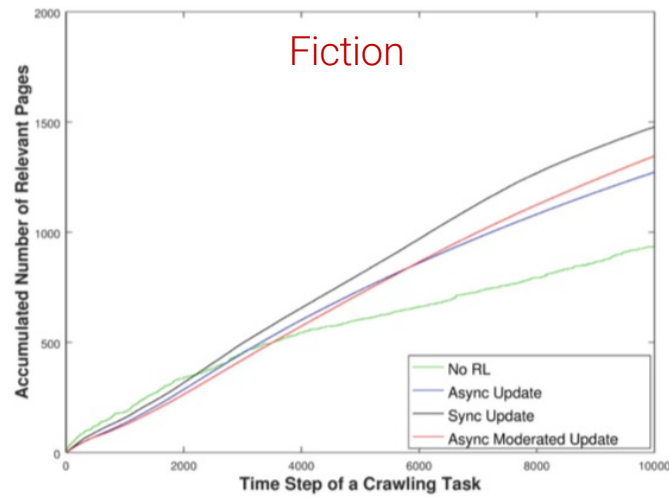
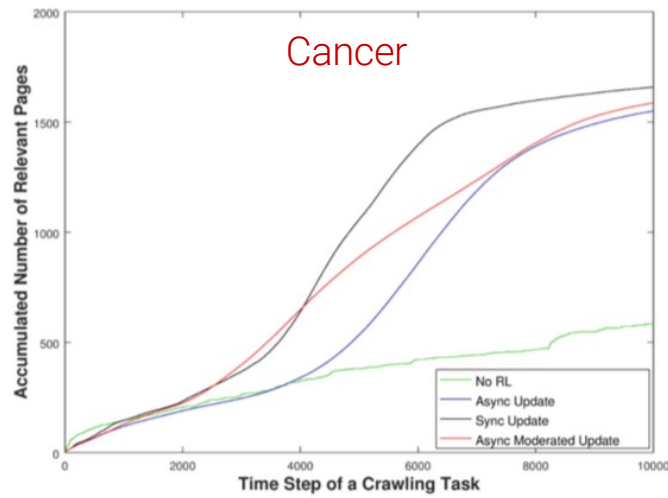
TD or Bellman error: difference between the action value in two successive states, t and t+1. "

Evaluation setting...

- Crawling task starts with a seed page and terminates when n pages are visited (n set to 10.000).
- At each time step, page is crawled and analyzed (cosine and word2vec similarity measures).
- Rewards:
 - 30 if TFIDS similarity > 0
 - 30 if TFIDF=0 and word2vec $> 0,5$
 - 20 if TFIDF=0 and word2vec $> 0,4$
 - -1, otherwise

Evaluation setting

- **Environment**: database dump of Simple English Wikipedia.
- **Target topics**:
 - Representative: Fiction, Olympics, Cancer
 - Sparse: Cameras, Geology, Poetry
- **Seed pages**: main pages of each category
- **Parameters**:
 - $\varepsilon = 0.1$, discount rate $\gamma = 0.9$, step size $\alpha = 0.001$.



Accumulated number of relevant pages per time step

Evaluation of crawlers

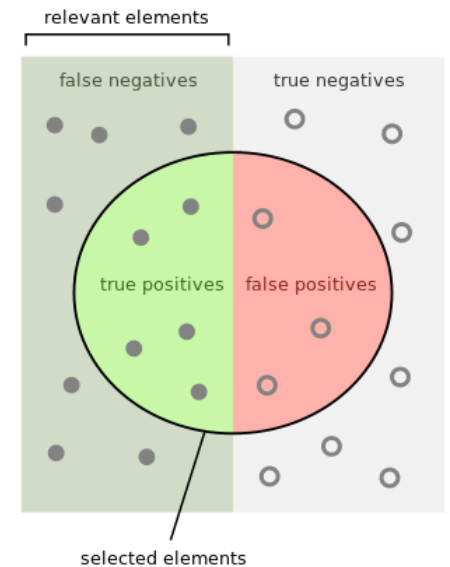
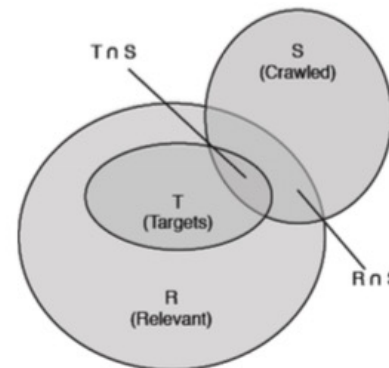
- How to compare crawling algorithms?
 - comparing applications
 - comparing algorithms
- Requirements:
 - sufficient number of runs over different topics
 - sound methodologies to consider temporal nature of results
- Challenges
 - unavailability of relevant sets for particular topics/queries.

Relevance estimation

- Estimation of page relevance (and quality)
 - number of keywords in the page
 - frequency of keywords
 - cosine similarity
 - classifier-based
 - *N* crawlers from same seeds until *p* pages retrieved
 - PageRank, HITS

Metrics

- Precision-like metrics:
 - harvest rate for Boolean relevance
 - average relevance for continuous relevance scores
- Search length
- Recall-like metrics:
 - without normalization
 - on predesigned crawling experiment (seeds selection)
- Use of critical resources



How many selected items are relevant?

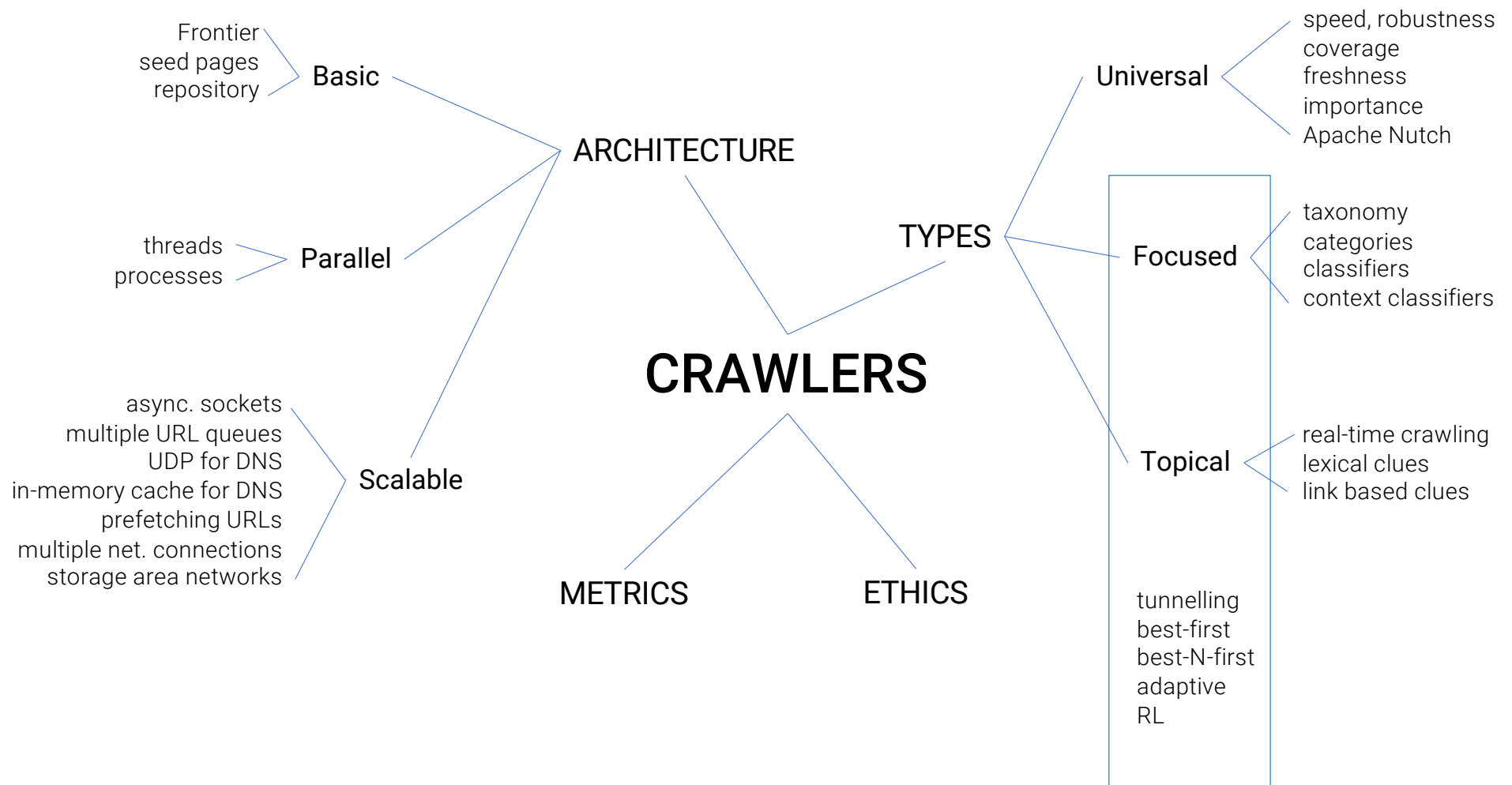
$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Crawler ethics

- Limiting server requests
- Distribution of request
- Crawler etiquette
- **User-Agent** HTTP and **From** header
- Robot Exclusion Protocol, **robots.txt**
 - User-agent: *
 - Disallow: /
- HTML meta tag



Module 2:

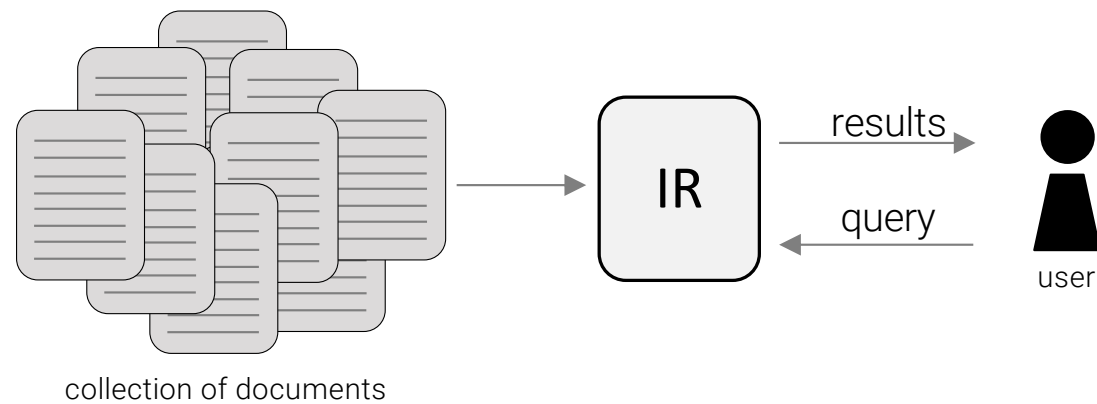
Structured data extraction

Module 3:

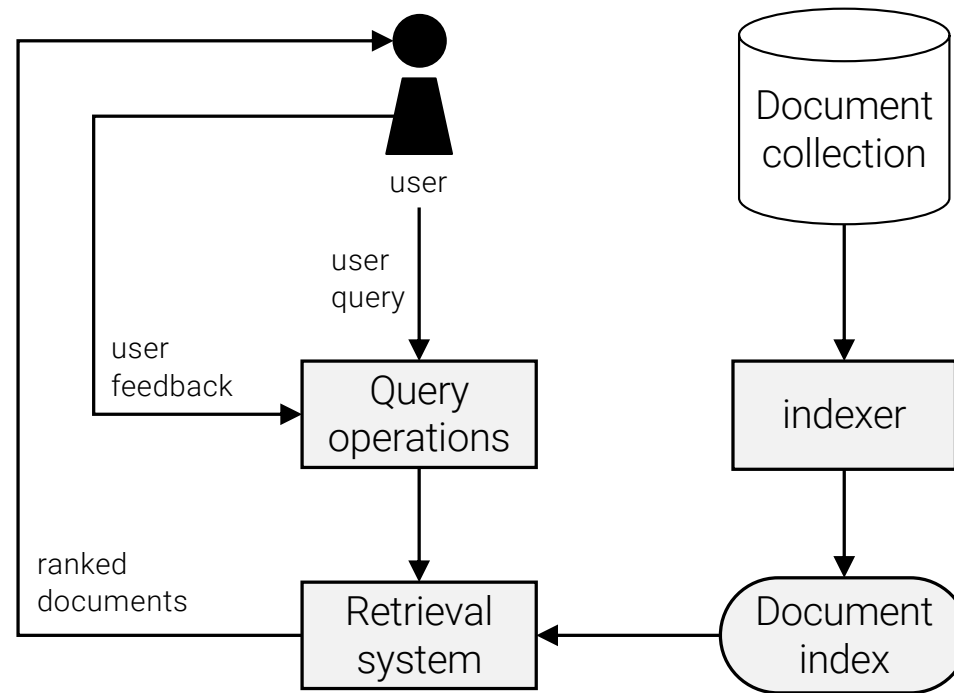
Information retrieval and web search

Information Retrieval

- Web search has roots in IR



IR general architecture



Information retrieval models

- IR model
 - how to represent documents and queries?
 - how to compute relevance of a document d to a query q ?
- Main IR models:
 - Boolean model,
 - vector space model,
 - language model,
 - probabilistic model.

IR model formalization

- D – collection of documents,
- $V = \{t_1, t_2, \dots, t_{|V|}\}$ – set of distinctive terms t_i in D – called vocabulary of D with size $|V|$,
- Each term t_i of a document $d_j \in D$ has a weight $w_{ij} > 0$.
- Each document d_j is represented with a **term vector**:
 $d_j = (w_{1j}, w_{2j}, \dots, w_{|V|j})$, where w_{ij} corresponds to $t_i \in V$ signifying the importance of t_i in d_j .

Boolean IR model

- Simplest IR model, queries and documents represented as sets of terms (with vectors):

$$q = \{t_i, t_p, t_j\}, \text{ where } t_i, t_p, t_j \in V$$

	t_1	t_2	...	t_i	...	t_p	...	t_j	...	$t_{ V }$
q	0	0	...	1	...	1	...	1	...	0

- Retrieval:
 - Exact matching based on Boolean algebra (AND, OR, NOT)

query: computer NOT science

Vector space model...

- Best known and widely used in IR
- Documents represented as **weight vectors**
- Variations of **TF/TF-IDF** used for weight calculation
- TF – term frequency:
 - $w_{ij} = f_{ij}$ where f_{ij} number of times t_i appears in d_j .
 - TF can be normalized, e.g. **Euclidean normalization**:

$$tf_{ij} = \frac{f_{ij}}{\sqrt{f_{1j}^2 + f_{2j}^2 + \dots + f_{|V|j}^2}}$$

TF-IDF scheme

- TF: non discriminative words (e.g. “the”, “this”,...) can have high weights.
- TF-IDF scheme relativizes terms appearing in many documents.
- Computed e.g. as $idf_i = \log \frac{N}{df_i}$
 - N total number of documents,
 - df_i the number of documents with t_i .
- According to TF-IDF, $w_{ij} = tf_{ij} * idf_i$

TF-IDF for queries

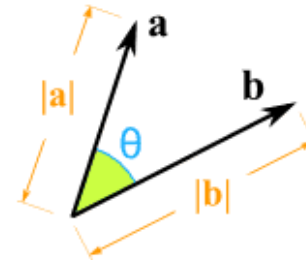
- Number of variations for calculating IDF exists.
- How about queries?
- Sulton and Buckley (1988)* suggests:

$$w_{ij} = \left(0.5 + \frac{0.5 * f_{ij}}{\max\{f_{1j}, f_{2j}, \dots, f_{|V|j}\}} \right) * \log \frac{N}{df_i}$$

*Salton, G. and C. Buckley. *Term-weighting approaches in automatic text retrieval*. Information Processing & Management, 1988, 24(5): p. 513-523.

Relevance ranking in vector space model

- Documents relevance ranking:
 - how relevant is $\mathbf{d}_j$ to $\mathbf{q}$?
 - many variants to measure relevance
- Dot product: $\text{sim}(\mathbf{d}_j, \mathbf{q}) = \langle \mathbf{d}_j \cdot \mathbf{q} \rangle$
- Cosine similarity:



$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| \times |\mathbf{b}| \times \cos(\theta)$$

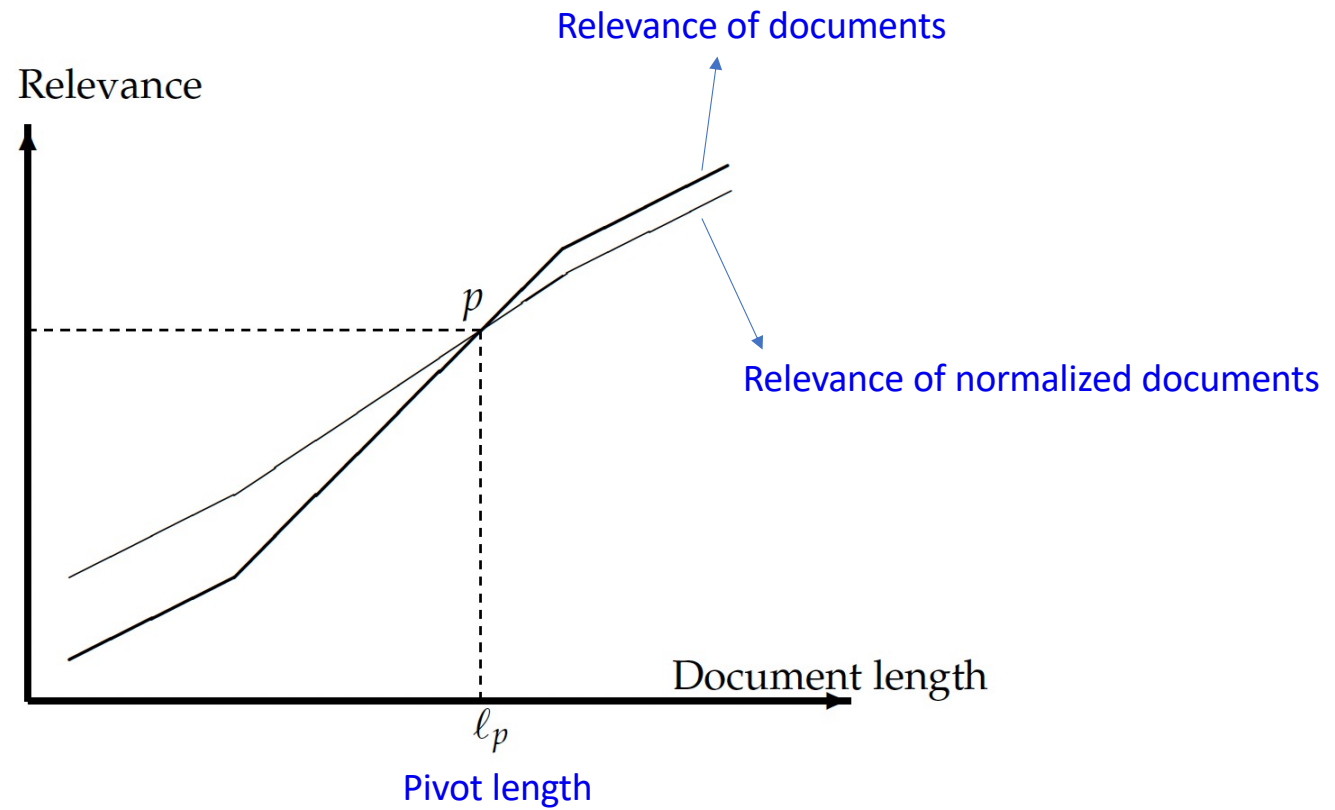
$$\mathbf{a} \cdot \mathbf{b} = a_x \times b_x + a_y \times b_y$$

$$\text{cosine}(\mathbf{d}_j, \mathbf{q}) = \frac{\langle \mathbf{d}_j \cdot \mathbf{q} \rangle}{\|\mathbf{d}_j\| * \|\mathbf{q}\|} = \frac{\sum_{i=1}^{|V|} w_{ij} * w_{iq}}{\sqrt{\sum_{i=1}^{|V|} w_{ij}^2} * \sqrt{\sum_{i=1}^{|V|} w_{iq}^2}}$$

Pivoted normalized document length

- Longer documents – more distinct terms, higher term frequencies
- Simple normalization by document length ... too crude:
 - without normalization, long documents can get unfairly high scores
 - with overly strong normalization, long documents can get unfairly penalized (multiple topics docs)
- Pivoted normalization: a balanced normalization: normalize document in a way that the effect of document length on relevance is not lost.
- Intuition: relevance does not grow linearly with document length.

D: document collection
Q: query ensemble

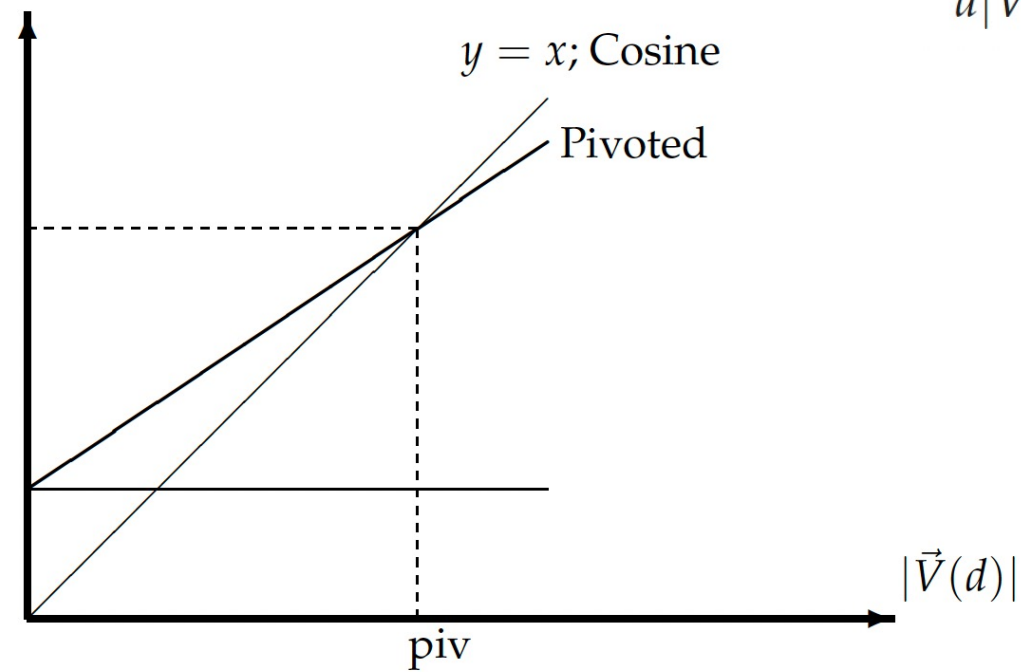


$$\text{score}(q, d) = \frac{\vec{V}(q) \cdot \vec{V}(d)}{|\vec{V}(q)| |\vec{V}(d)|}$$

$$a|\vec{V}(d)| + (1 - a)\text{piv.}$$

$$au_d + (1 - a)\text{piv,}$$

Pivoted normalization



Statistical language model

- Based on **probability**, funded on **statistical theory**.
- Idea:
 - for each document d estimate a **language model**
 - rank documents by the likelihood of the query q given the language model.
- Similar approaches used in **NLP** and **speech recognition**.
- First proposed by Ponte and Croft*

*Ponte, J. and W. Croft. *A language modeling approach to information retrieval*. In Proceedings of ACM SIGIR Conf. on Research and Development in Information Retrieval (SIGIR-1998), 1998..

Formalization

- We are interested in estimating $\Pr(q|d_j)$, where
 - $q = q_1 q_2 \dots q_m$ is a query
 - D is document collection and $d_j \in D$
- Typically based on unigrams
- Better models consider n-grams

Multinomial distribution

- Multinomial distribution over unigrams (words):

$$Pr(q|d_j) = \prod_{i=1}^m Pr(q_i|d_j) = \prod_{i=1}^{|V|} Pr(t_i|d_j)^{f_{iq}}$$

- where

- $q = q_1 q_2 \dots q_m$,
- V is vocabulary of d_j
- $t_i \in V$ and f_{iq} tells the number of times t_i appears in q
- $Pr(t_i|d_j) = \frac{f_{ij}}{|d_j|}$ and $\sum_{i=1}^{|V|} Pr(t_i|d_j) = 1$

Laplace and Lindstone smoothing

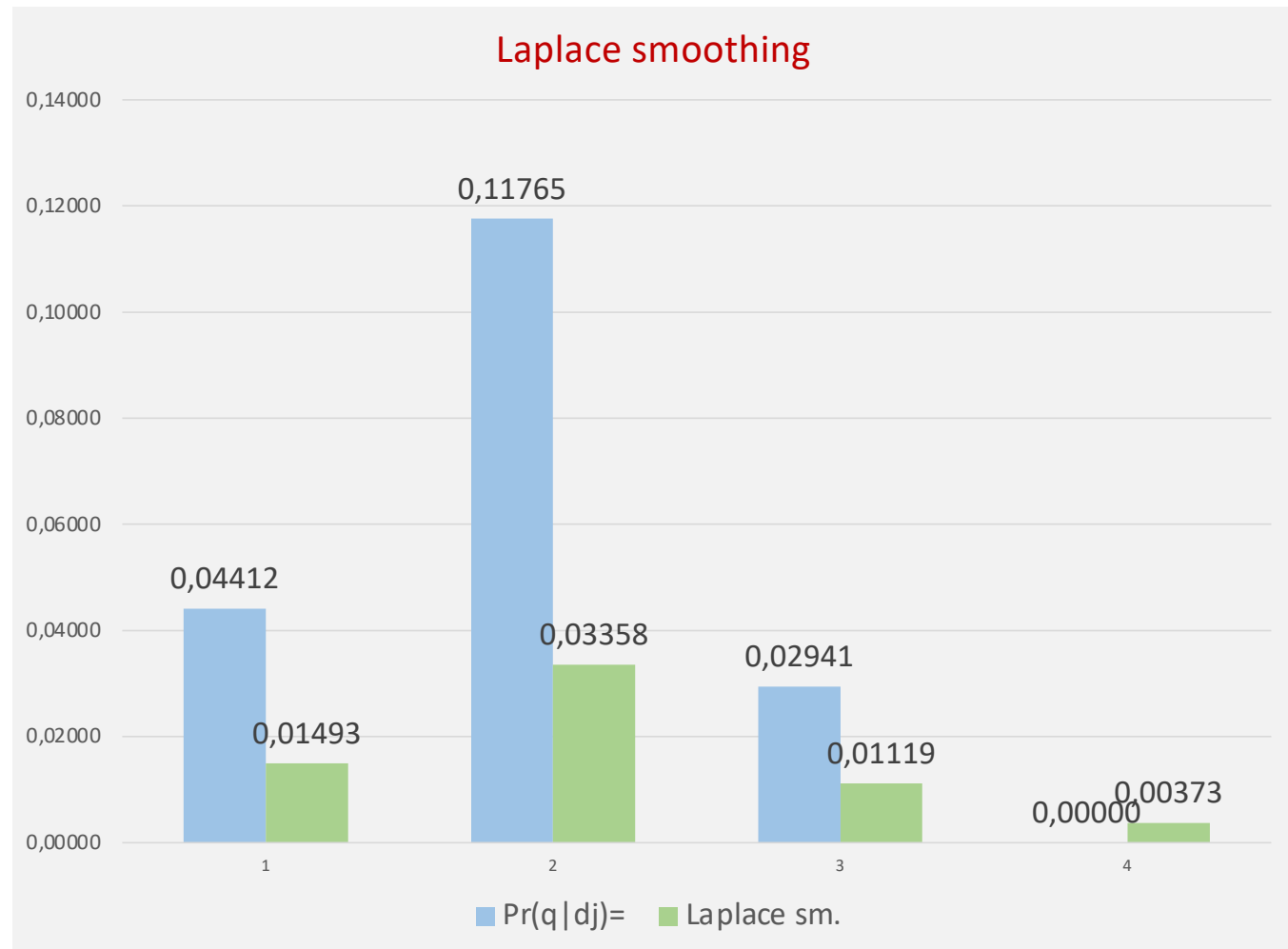
- What if $t_i \in q$ and $t_i \notin d_j \Rightarrow \Pr(t_i|d_j) = 0$?
- Additive smoothing:

$$Pr_{add}(t_i|d_j) = \frac{\lambda + f_{ij}}{\lambda|V| + |d_j|}$$

- Laplace smoothing: $\lambda = 1$
- Lindstone smoothing: $0 < \lambda < 1$

Example

- $q = t_1 t_2 t_3 t_4$
- $|V| = 200$
- $|d_j| = 68$
- $f_{1j} = 3$
- $f_{2j} = 8$
- $f_{3j} = 2$
- $f_{4j} = 0$



Okapi method – BM25

- Variation of a relevance measure, **Okapi method***:

$$okapi(d_j, q) = \sum_{t_i \in q, d_j} \ln \frac{N - df_i + 0.5}{df_i + 0.5} * \frac{(k_1 + 1)f_{ij}}{k_1 \left(1 - b + b \frac{dl_j}{avdl}\right) + f_{ij}} * \frac{(k_2 + 1)f_{iq}}{k_2 + f_{iq}}$$

- where:

- df_i is the number of documents that contain the term t_i
- dl_j is the document length (in bytes) of d_j
- $avdl$ is the average document length of the collection
- Parameters: k_1 (between 1.0-2.0), b (usually 0.75) and k_2 (between 1-1000).

*Robertson, S., S. Walker, and M. Beaulieu. *Okapi at TREC-7: automatic ad hoc filtering*, VLC and interactive track. NIST Special Publications, 1999: p. 253-264.

Relevance feedback

- Improving IR based on user's feedback
- Rocchio method:
 - let $\mathbf{q}$ be original query vector and
 - D_r the set of relevant and D_{ir} the set of irrelevant documents.
 - The expanded query $\mathbf{q}_e$ can be calculated as:

$$\mathbf{q}_e = \alpha \mathbf{q} + \frac{\beta}{|D_r|} \sum_{d_r \in D_r} \mathbf{d}_r - \frac{\gamma}{|D_{ir}|} \sum_{d_{ir} \in D_{ir}} \mathbf{d}_{ir}$$

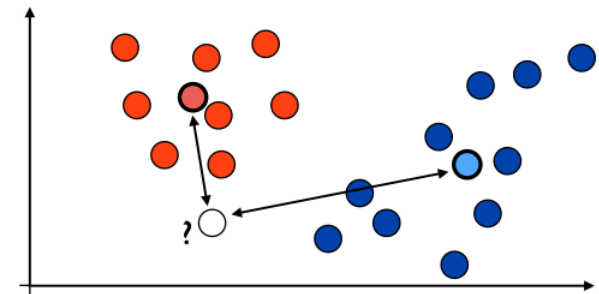
- where α , β and γ parameters

Rocchio classification

- Supervised machine learning on feedback – no comparison with query.
- Prototype vectors c_i built for each class:

$$c_i = \frac{\alpha}{|D_i|} \sum_{d \in D_i} \frac{d}{||d||} - \frac{\beta}{|D - D_i|} \sum_{d \in D - D_i} \frac{d}{||d||}$$

- where:
 - D_i set of documents of class i
 - α and β parameters



Other ML approaches

- LU learning
- PU learning
- Ranking SVM*
- Pseudo-relevance feedback

*Joachims, T. Optimizing search engines using clickthrough data. In Proceedings of ACM SIGKDD International Conference on Knowledge, Discovery and Data Mining (KDD-2002), 2002.

Word embeddings

- Representation of words in **high-dimensional space**
 - e.g. 300 – 500 dimensions
 - words with similar semantics/usage appear closer...

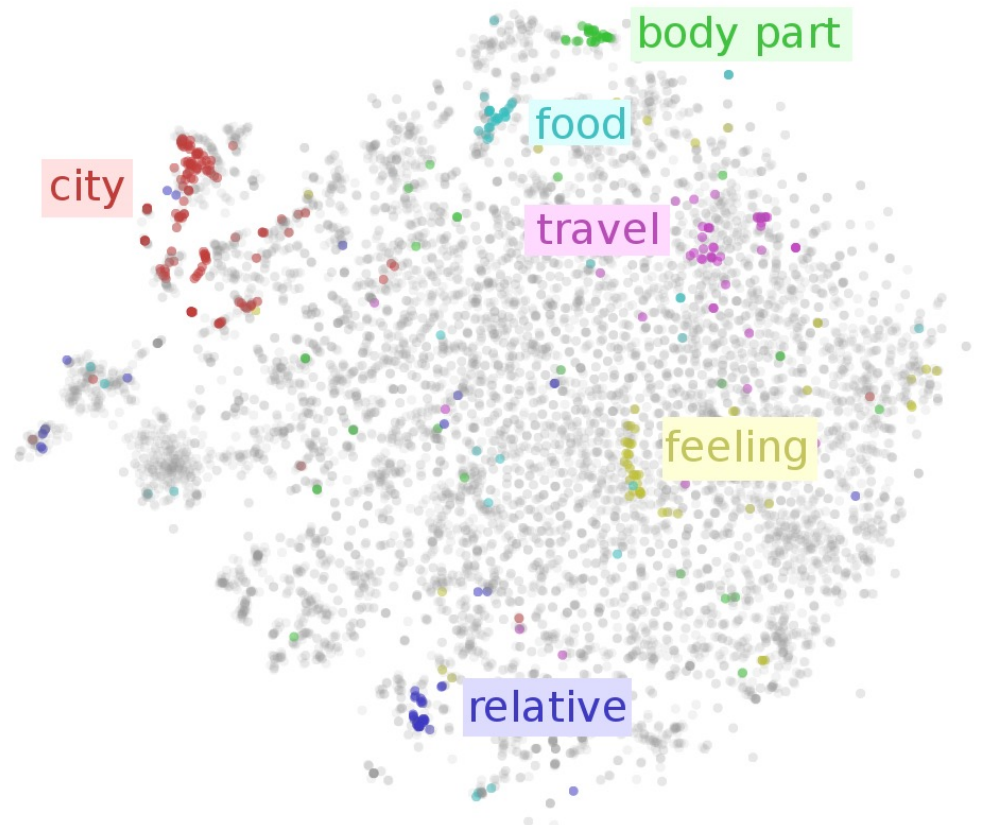


Image source: <http://ruder.io/word-embeddings-1/>

© Laboratory for Data Technologies, UL FRI

One hot encoding

- Encoding words with vocabulary words
 - vocabulary = {house, real-estate, rent, agency, provision, flat}
- One hot encoding

Word	v1	v2	v3	v4	v5	v6
house	1	0	0	0	0	0
real-estate	0	1	0	0	0	0
rent	0	0	1	0	0	0
agency	0	0	0	1	0	0
provision	0	0	0	0	1	0
flat	0	0	0	0	0	1

Document	v1	v2	v3	v4	v5	v6
real-estate agency	0	1	0	1	0	0

One hot encoding in vector space

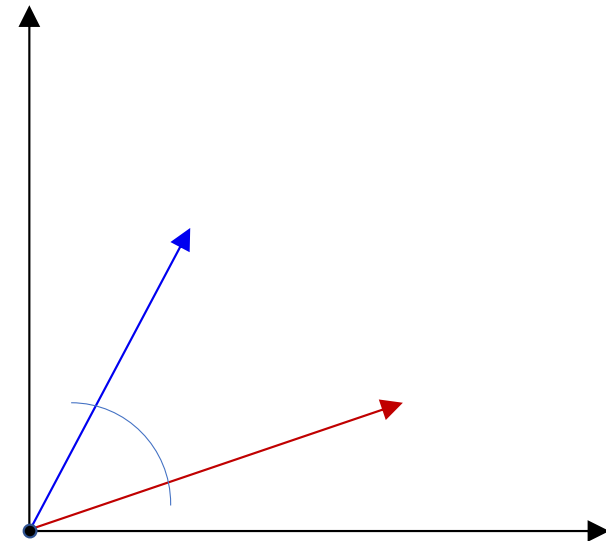
■ Corpus:

- Romeo and Juliet.
- Juliet: O happy dagger!
- ...

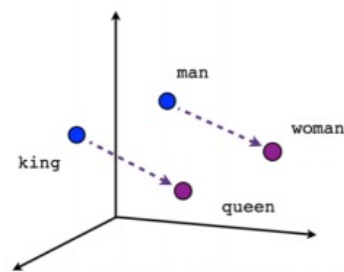
■ Vocabulary:

- $V = \{\text{romeo}, \text{juliet}, \text{happy}, \text{dagger}, \dots\}$,

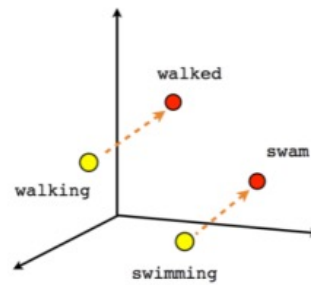
Category	w_1	w_2	...	$w_{ V }$
romeo	1	0	...	0
juliet	0	1	...	0
...				



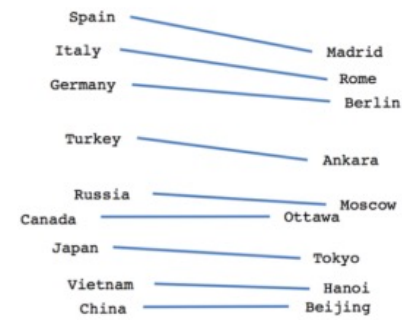
Word embeddings in vector space



Male-Female



Verb tense



Country-Capital

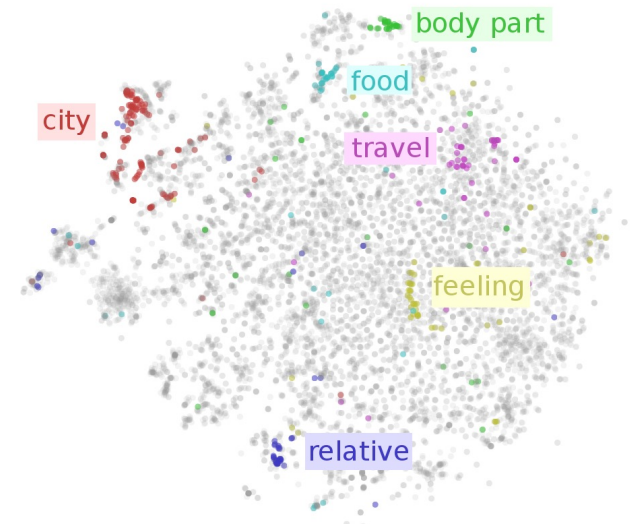
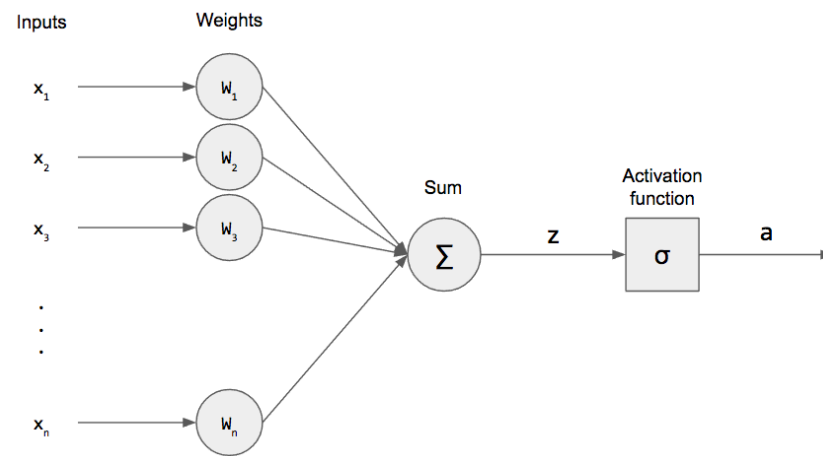


Image source: <https://cbail.github.io/texasdata/word2vec/rmarkdown/word2vec.html>

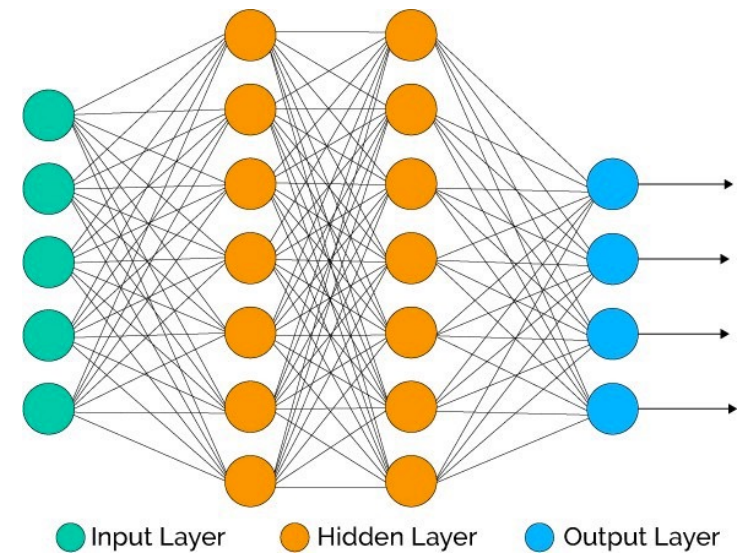
© Laboratory for Data Technologies, UL FRI

Neural networks

■ Artificial neuron

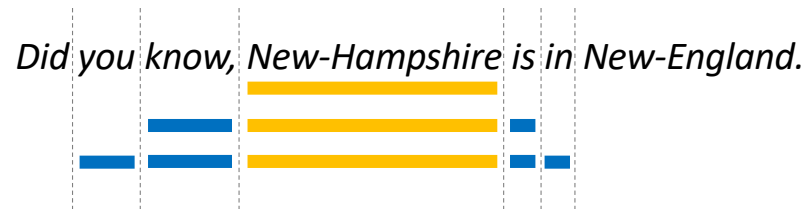


■ Neural network

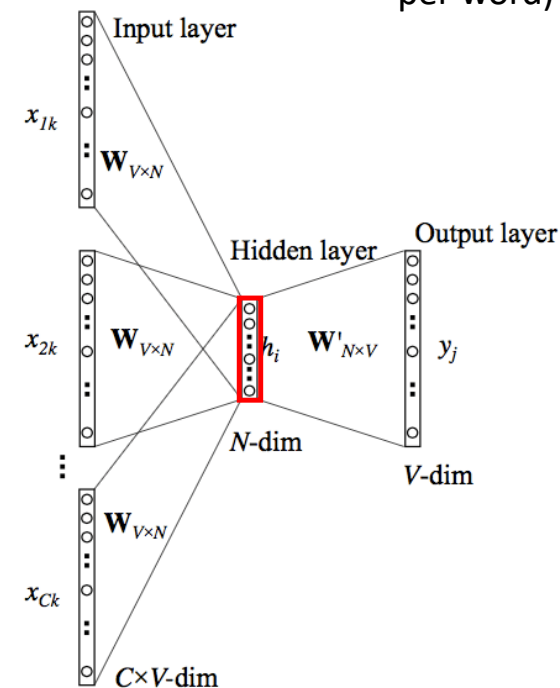


Learning word embeddings

- Co-occurring words
- Context window

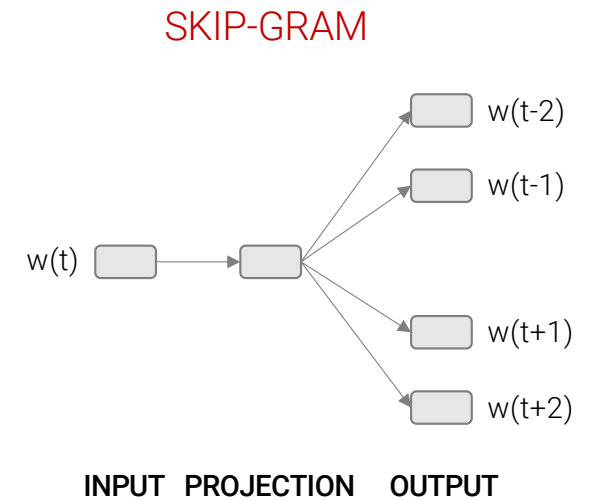
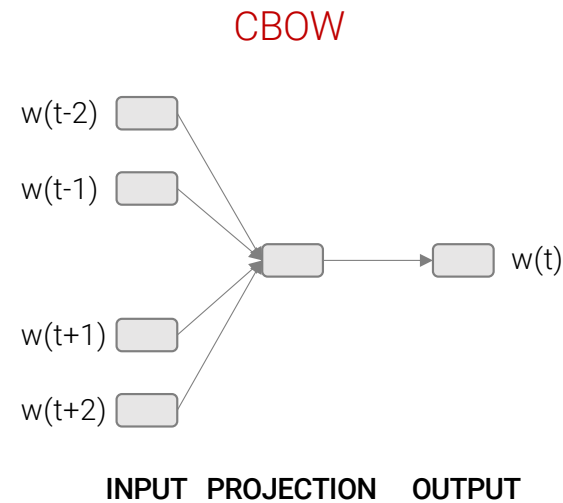


$W_{V \times N}$ is a shared matrix for all words in the V (one row per word)



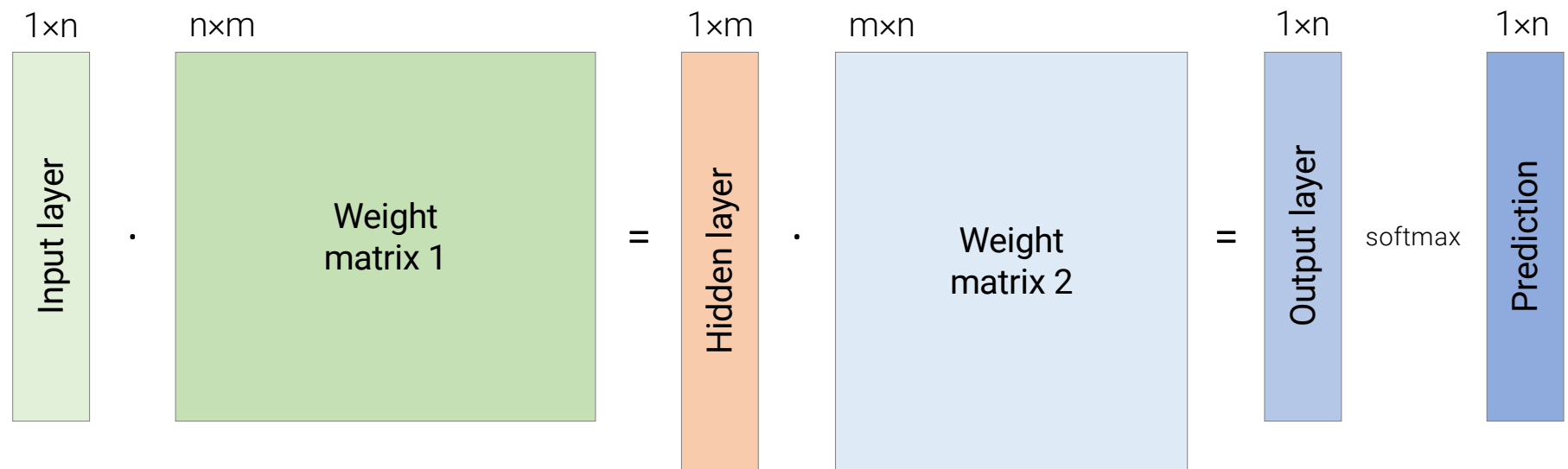
Embeddings models

- CBOW
- Skip-Gram



Learning word embeddings with NN

- Shallow NN



Word2Vec*

- Converts words to vectors using CBOW or SKIP-GRAM model implemented with shallow NN.
- Input:
 - text corpus
- Output:
 - word vector file

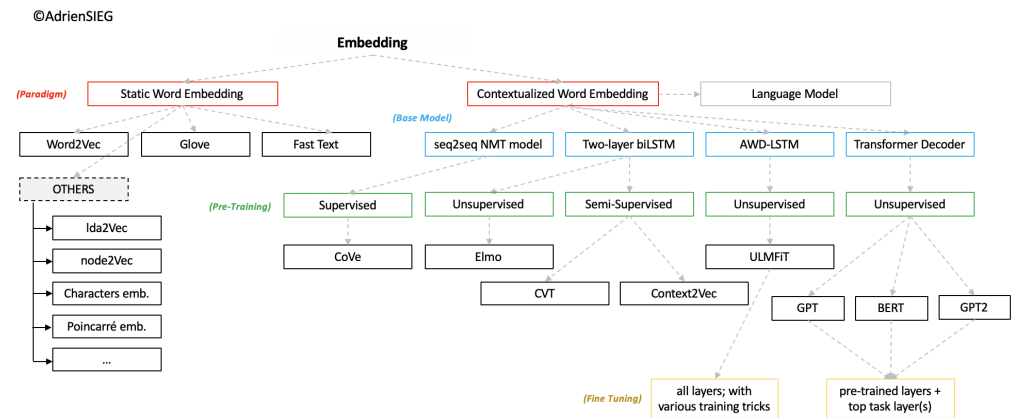
*T. Mikolov, K. Chen, G. Corrado, and J. Dean. *Efficient Estimation of Word Representations in Vector Space*. ArXiv e-prints, January 2013.

Other approaches

- Glove (Stanford, 2013)¹
- FastText (Facebook, 2016)²
- ELMO
- BERT

¹ <https://nlp.stanford.edu/projects/glove/>

² <https://fasttext.cc/>



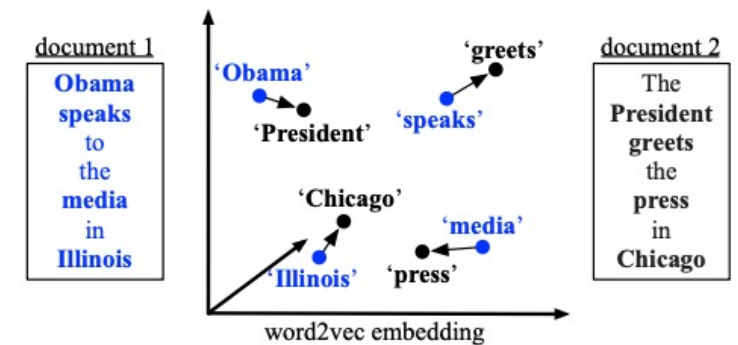
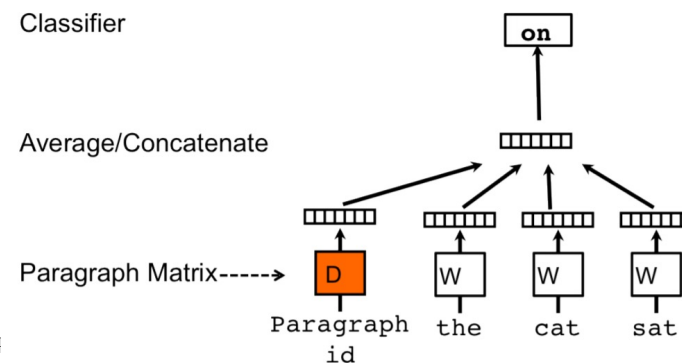
GloVe - Global Vectors for Word Representation

- The core idea is:
 - Build a **co-occurrence matrix** X , where X_{ij} = how often word w_j appears in the context of word w_i .
 - Learn word vectors so that their dot product matches the **log of that count**.
 - The main GloVe equation is:

$$w_i^T \tilde{w}_j + b_i + \tilde{b}_j \approx \log(X_{ij})$$

Using word embeddings in IR

- Word embeddings useful to retrieve similar words
- How to retrieve documents similar to a specific query?
- Approaches:
 - Weighted average of word embeddings
 - Word Mover's Distance (WMD)
 - Doc2Vec



WMD example

■ Interpretation

- WMD treats each document as a distribution of word mass and asks: what is the cheapest way to move the mass of d_1 onto d_2 using distances between word embeddings?



WMD – normalizing masses

Documents

$d_1 = [\text{dog, barks, loudly, at, night}]$

$d_2 = [\text{puppy, howls, at, night, very, loudly}]$

Mass normalization (normalized TF) $mass(t_i, d_j) = \frac{tf_i}{\sum_{k=1}^m tf_k}$

Each word in d_1 appears once $\rightarrow$ each word in d_1 gets mass $= \frac{1}{5} = 0.2$

Each word in d_2 appears once $\rightarrow$ each word in d_2 gets mass $= \frac{1}{6} \approx 0.1667$

WMD – cost matrix

In WMD, the cost $c(i, j)$ is usually the Euclidean distance between the embedding of word i from d_1 and word j from d_2 :

$$c(i, j) = \|X_i - X_j\|$$

	puppy	howls	at	night	very	loudly
dog	1.0	3.0	4.0	4.0	5.0	4.0
barks	3.0	1.0	4.0	4.0	5.0	2.0
loudly	4.0	3.0	4.0	4.0	1.5	0.0
at	4.0	4.0	0.0	2.0	4.0	4.0
night	4.0	4.0	2.0	0.0	4.0	4.0

These numbers are illustrative. In practice they come from actual distances between pretrained word embeddings.

Close semantic matches

dog → puppy
barks → howls

Exact matches

at → at
night → night
loudly → loudly

Transport plan T

A transport plan is valid if every row sums to the mass in d_1 and every column sums to the mass in d_2 .

$$WMD(d_1, d_2) = \sum_{i,j} T_{ij} C_{ij}$$

	puppy	howls	at	night	very	loudly	
dog	0.1667					0.0333	0.2
barks		0.1667				0.0333	0.2
loudly					0.1667	0.0333	0.2
at			0.1667			0.0333	0.2
night				0.1667		0.0333	0.2
	0.1667	0.1667	0.1667	0.1667	0.1667	0.1667	

Final WMD

Nonzero contributions:

dog: $0.1667 * 1.0 + 0.0333 * 4.0 = 0.2999$

barks: $0.1667 * 1.0 + 0.0333 * 2.0 = 0.2333$

loudly: $0.1667 * 1.5 + 0.0333 * 0.0 = 0.2501$

at: $0.1667 * 0.0 + 0.0333 * 4.0 = 0.1332$

night: $0.1667 * 0.0 + 0.0333 * 4.0 = 0.1332$

	puppy	howls	at	night	very	loudly
dog	1.0	3.0	4.0	4.0	5.0	4.0
barks	3.0	1.0	4.0	4.0	5.0	2.0
loudly	4.0	3.0	4.0	4.0	1.5	0.0
at	4.0	4.0	0.0	2.0	4.0	4.0
night	4.0	4.0	2.0	0.0	4.0	4.0

	puppy	howls	at	night	very	loudly	
dog	0.1667					0.0333	0.2
barks		0.1667				0.0333	0.2
loudly					0.1667	0.0333	0.2
at			0.1667			0.0333	0.2
night				0.1667		0.0333	0.2
	0.1667	0.1667	0.1667	0.1667	0.1667	0.1667	

$$WMD(d_1, d_2) = \sum_{i,j} T_{ij} C_{ij} \approx 1.05$$

Dual Embedding Space Model

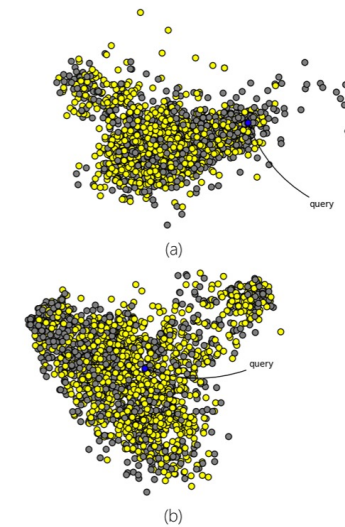
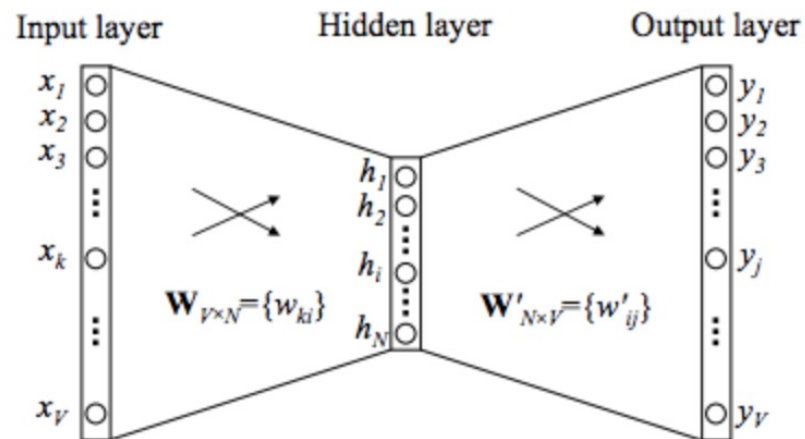
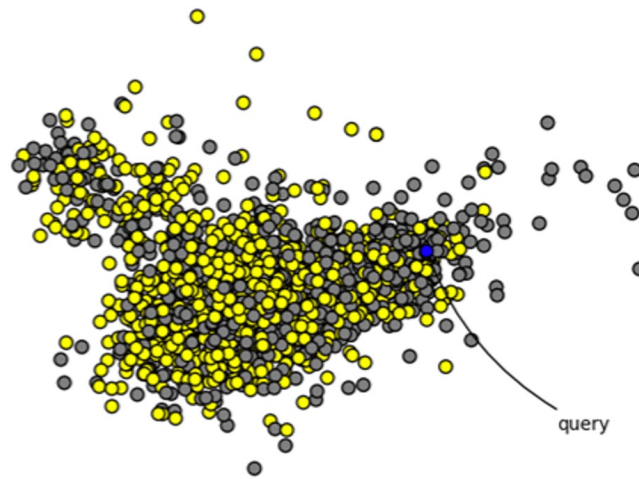
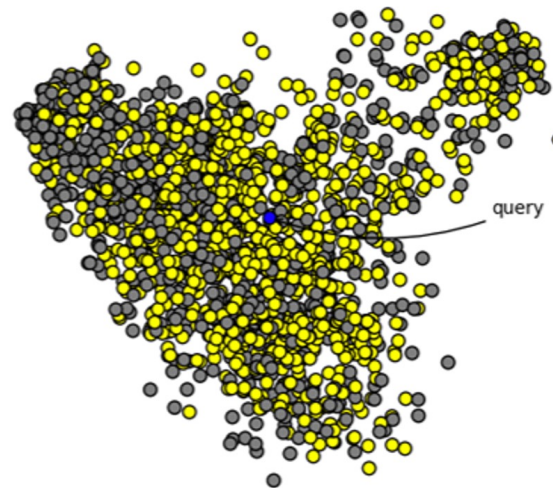


Figure 1: A two dimensional PCA projection of the 200-dimensional embeddings. Relevant documents are yellow, irrelevant documents are grey, and the query is blue. To visualize the results of multiple queries at once, before dimensionality reduction we centre query vectors at the origin and represent documents as the difference between the document vector and its query vector. (a) uses IN word vector centroids to represent both the query and the documents. (b) uses IN for the queries and OUT for the documents, and seems to have a higher density of relevant documents near the query.

*B. Mitra, E. Nalisnick, N. Craswell and R. Caruana. *A Dual Embedding Space Model for Document Ranking*. arXiv:1602.01137v1 [cs.IR] 2 Feb 2016.



(a)



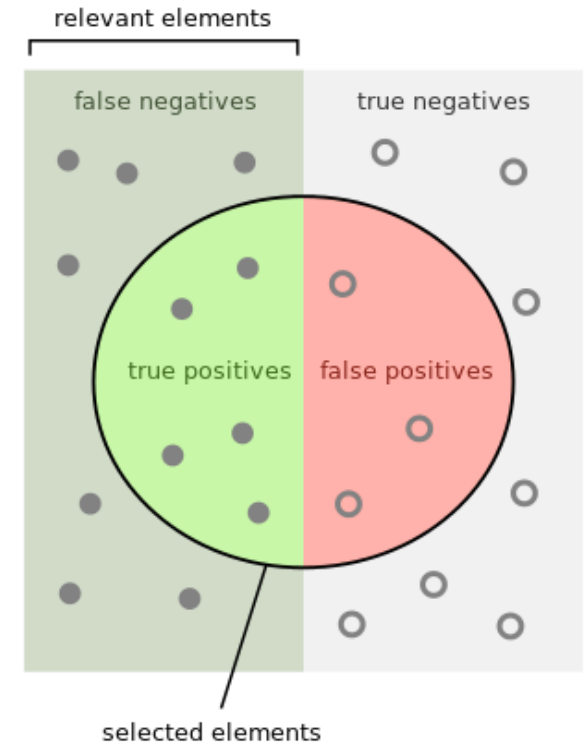
(b)

Table 1: The nearest neighbours for the words "yale", "seahawks" and "eminem" according to the cosine similarity based on the IN-IN, OUT-OUT and IN-OUT vector comparisons for the different words in the vocabulary. These examples show that IN-IN and OUT-OUT cosine similarities are high for words that are similar by function or type (*typical*), and the IN-OUT cosine similarities are high between words that often co-occur in the same query or document (*topical*). The *word2vec* model used here was trained on a query corpus with a vocabulary of 2,748,230 words.

yale			seahawks			eminem		
IN-IN	OUT-OUT	IN-OUT	IN-IN	OUT-OUT	IN-OUT	IN-IN	OUT-OUT	IN-OUT
yale	yale	yale	seahawks	seahawks	seahawks	eminem	eminem	eminem
harvard	uconn	faculty	49ers	broncos	highlights	rihanna	rihanna	rap
nyu	harvard	alumni	broncos	49ers	jerseys	ludacris	dre	featuring
cornell	tulane	orientation	packers	nfl	tshirts	kanye	kanye	tracklist
tulane	nyu	haven	nfl	packers	seattle	beyonce	beyonce	diss
tufts	tufts	graduate	steelers	steelers	hats	2pac	tupac	performs

Evaluation measures

- In ML, we use **precision** and **recall**
- In IR and web search we **rank** not classify!



How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

source: Wikipedia

Evaluation of ranking

- Let D be a collection of documents with $|D| = N$.
- For a query q the retrieval algorithm produces ranking $R_q: \langle d_1^q, d_2^q, \dots, d_N^q \rangle$, where $d_1^q \in D$ most relevant and $d_N^q \in D$ most irrelevant document to q .
- P and R can be calculated for each rank position:
 - Recall at rank position $i: r(i) = \frac{S_i}{|D_q|}$, $D_q \subseteq D$: set of actual relevant documents
 - Precision at rank position $i: p(i) = \frac{S_i}{i}$, S_i : number of relevant documents from position 1 to i .

Example of ranking at each position

Rank i	+/-	p(i)	r(i)
1	+	$1/1 = 100\%$	$1/8 = 13\%$
2	+	$2/2 = 100\%$	$2/8 = 25\%$
3	+	$3/3 = 100\%$	$3/8 = 38\%$
4	-	$3/4 = 75\%$	$3/8 = 38\%$
5	+	$4/5 = 80\%$	$4/8 = 50\%$
6	-	$4/6 = 67\%$	$4/8 = 50\%$
7	+	$5/7 = 71\%$	$5/8 = 63\%$
8	-	$5/8 = 63\%$	$5/8 = 63\%$
9	+	$6/9 = 67\%$	$6/8 = 75\%$
10	+	$7/10 = 70\%$	$7/8 = 88\%$
11	-	$7/11 = 64\%$	$7/8 = 88\%$
12	+	$8/12 = 67\%$	$8/8 = 100\%$

Average precision

- Average precision:

Rank i	+/-	p(i)	r(i)
1	+	1/1 = 100%	1/8 = 13%
2	+	2/2 = 100%	2/8 = 25%
3	+	3/3 = 100%	3/8 = 38%
4	-	3/4 = 75%	3/8 = 38%
5	+	4/5 = 80%	4/8 = 50%
6	-	4/6 = 67%	4/8 = 50%
7	+	5/7 = 71%	5/8 = 63%
8	-	5/8 = 63%	5/8 = 63%
9	+	6/9 = 67%	6/8 = 75%
10	+	7/10 = 70%	7/8 = 88%
11	-	7/11 = 64%	7/8 = 88%
12	+	8/12 = 67%	8/8 = 100%

$$p_{avg} = \frac{\sum_{d_i^q \in D_q} p(i)}{|D_q|}$$

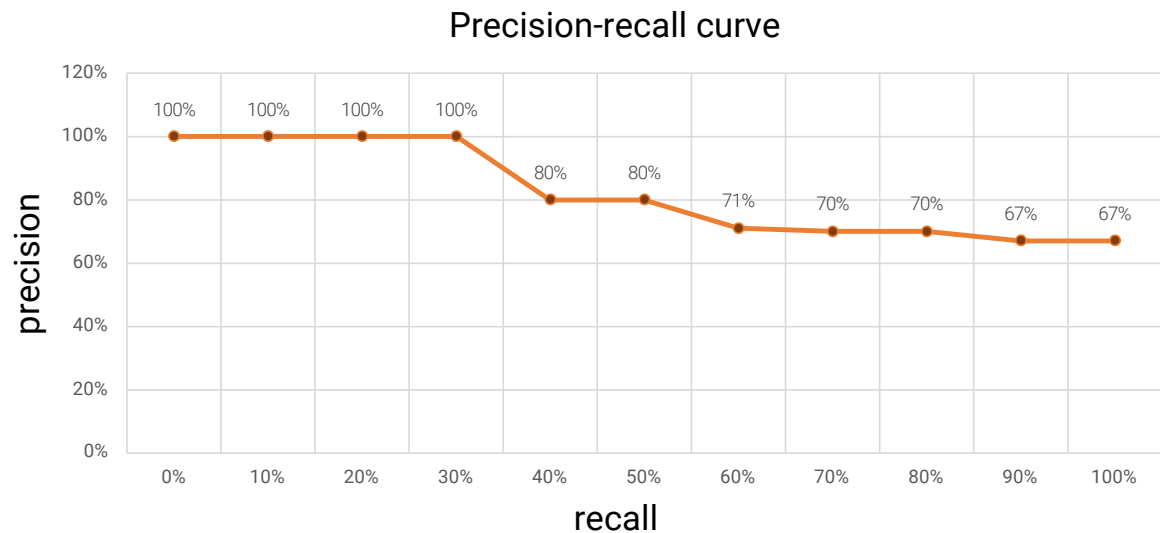
$$p_{avg} = \frac{100\% + 100\% + 100\% + 80\% + 71\% + 67\% + 70\% + 67\%}{8} = 82\%$$

Precision-recall curve

- Plotted at standard recall levels: 0%, 10%, 20%, ..., 100%

- $p(r_i) = \max_{r_i \leq r \leq r_{10}} p(r)$

i	Relevant	$p(r_i)$	r_i
1	+	100%	13%
2	+	100%	25%
3	+	100%	38%
4	-	75%	38%
5	+	80%	50%
6	-	67%	50%
7	+	71%	63%
8	-	63%	63%
9	+	67%	75%
10	+	70%	88%
11	-	64%	88%
12	+	67%	100%



Evaluation on multiple queries

- Overall precision on multiple queries:

$$\bar{p}(r_i) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} p_j(r_i)$$

- where:
 - $|Q|$ number of queries,
 - $p_j(r_i)$ precision q_j at the recall level r_i

Inverted index

- Most efficient index for IR
- Consists of:
 - vocabulary V of distinctive terms of the document set $D = \{d_1, d_2, \dots, d_n\}$
 - for each term t_i an inverted index of **postings**.

$$t_i \rightarrow \langle \text{posting}_1 \rangle, \langle \text{posting}_2 \rangle, \dots, \langle \text{posting}_n \rangle$$

- **Posting**: doc id + other relevant information for retrieval

Example

- Example of inverted index for proximity search:
- $\langle id_j, f_{ij}, [o_1, o_2, \dots, o_{|f_{ij}|}] \rangle$ where
 - f_{ij} - frequency of t_i in d_j
 - o_i - offset of t_i in d_j

Corpus:

id_1 : Web mining is useful.
 1 2 3 4

id_2 : Usage mining applications.
 1 2 3

id_3 : Web structure mining studies the Web hyperlink structure.
 1 2 3 4 5 6 7 8

Vocabulary:

{web, mining, useful, applications, usage, structure, studies, hyperlink}

Index:

- applications: $\langle id_2, 1, [3] \rangle$
- hyperlink: $\langle id_3, 1, [7] \rangle$
- mining: $\langle id_1, 1, [2] \rangle, \langle id_2, 1, [2] \rangle, \langle id_3, 1, [3] \rangle$
- structure: $\langle id_3, 2, [2, 8] \rangle$
- studies: $\langle id_3, 1, [4] \rangle$
- usage: $\langle id_2, 1, [1] \rangle$
- useful: $\langle id_1, 1, [4] \rangle$
- web: $\langle id_1, 1, [1] \rangle, \langle id_3, 2, [1, 6] \rangle$

Search on inverted index

- Algorithm:

```
for each query term  $q_i \in Q$ 
  find  $q_i$  in vocabulary  $V$ 
  retrieve list  $list_i$  of docs  $d_j$  where  $d_j \in D$  and  $q_i \in d_j$ 
end
merge doc lists  $list_i \rightarrow$  documents that contain all the query terms
for each doc  $d_j \in merge\_list$ 
  compute rank score
end
print ranked list of docs
```

Example

- Which document gets the highest rank in the proximity search for $Q = \text{web mining}$?

Corpus:

id_1 : Web mining is useful.

id_2 : Usage mining applications.

id_3 : Web structure mining studies the Web hyperlink structure.

Index:

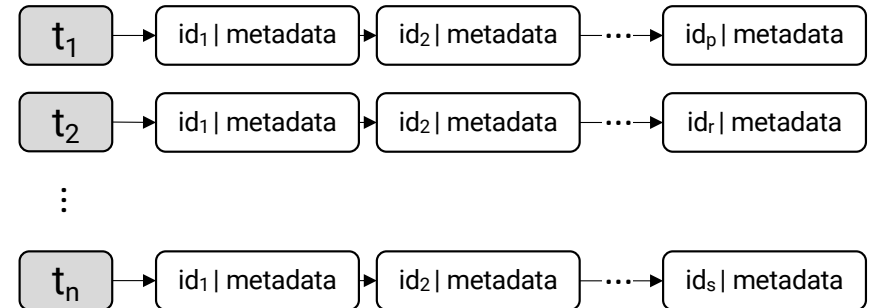
- applications: $\langle id_2, 1, [3] \rangle$
- hyperlink: $\langle id_3, 1, [7] \rangle$
- mining: $\langle id_1, 1, [2] \rangle, \langle id_2, 1, [2] \rangle, \langle id_3, 1, [3] \rangle$
- structure: $\langle id_3, 2, [2, 8] \rangle$
- studies: $\langle id_3, 1, [4] \rangle$
- usage: $\langle id_2, 1, [1] \rangle$
- useful: $\langle id_1, 1, [4] \rangle$
- web: $\langle id_1, 1, [1] \rangle, \langle id_3, 2, [1, 6] \rangle$

Query: web mining

Index construction

■ Algorithm:

```
for each doc  $d_j \in D$ 
  for each term  $t \in d_j$ 
    find position  $e$  of  $t$  in index
    if  $t$  not found then create new element  $e$  in index
    add (ID, metadata) of  $d_j$  to index at position  $e$ 
  end
end
```

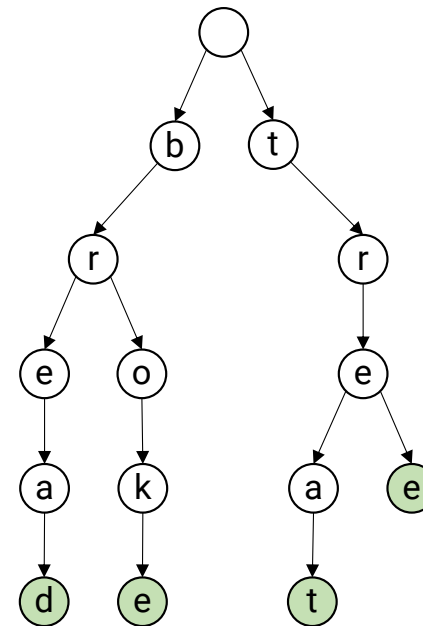


Trie data structure

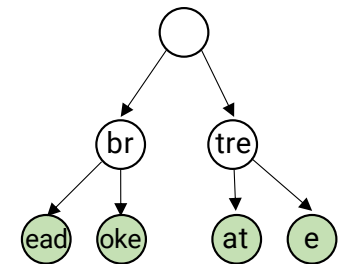
- Trie – retrieval
- Prefix tree with ordered content
- Nodes: string characters
- Example:
 - words: bread, broke, treat, tree
- Visualization:

<https://www.cs.usfca.edu/~galles/visualization/Trie.html>

Full representation



Compact representation



Example

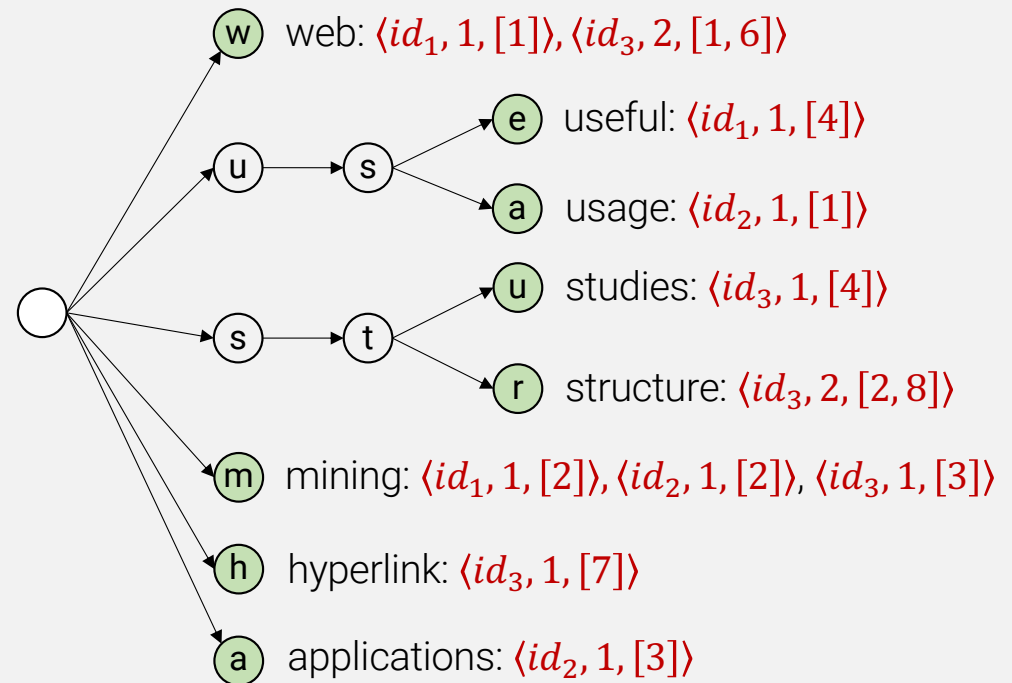
- Build trie data structure for corpus: id_1 , id_2 , id_3
- Vocabulary: {applications, hyperlink, mining, structure, studies, usage, useful, web}

Corpus:

id_1 : Web mining is useful.
1 2 3 4

id_2 : Usage mining applications.
1 2 3

id_3 : Web structure mining studies the Web hyperlink structure.
1 2 3 4 5 6 7 8



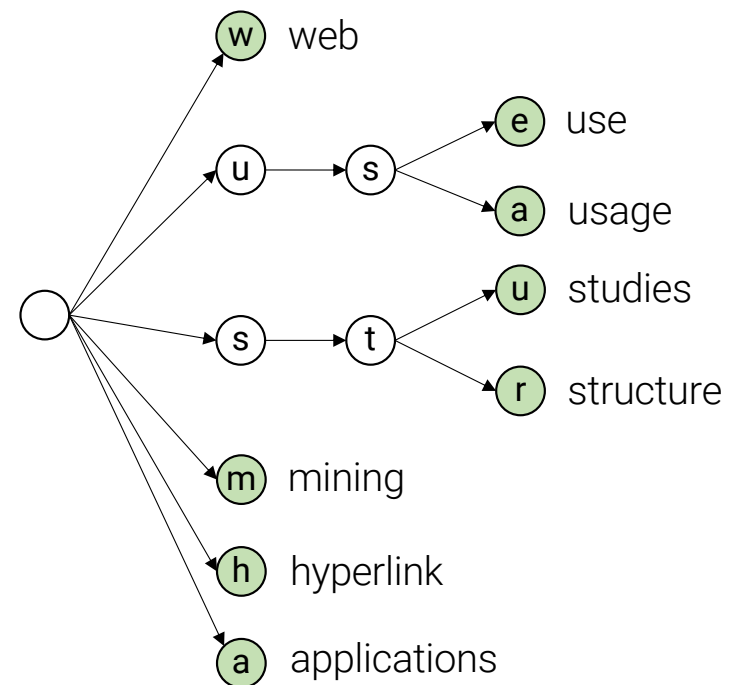
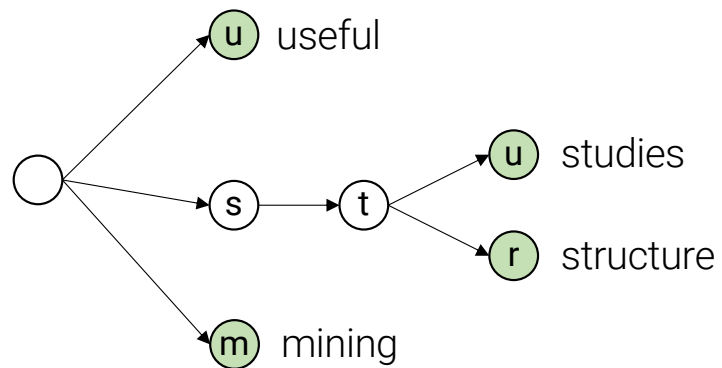
Time complexity

- Trie construction
 - Linear: $O(T)$, where T number of terms including duplicates
- Trie search
 - Linear: $O(n)$, where n length of a retrieved term

Trie construction for Web

- In Web environment, trie becomes very large.
- Changes in the construction algorithm:
 - write to a partial index I_i
 - when memory full, write partial index to disk
 - continue with new partial index
 - when all documents processed, merge indices:
 - pairwise (i.e. I_{1-2} , I_{3-4} , ..., I_{n-n+1} then I_{12-34} , ...) or
 - step by step (i.e. I_{1-2} , I_{12-3} , I_{123-4} , ...)

Trie merging



Additional indices

- Web dynamic, pages constantly added, deleted
- Additional indices:
 - index of added pages
 - index of deleted pages
- Algorithm:
 - search in main index $\rightarrow D_0$
 - search in index of added pages $\rightarrow D_+$
 - search in index of deleted pages $\rightarrow D_-$
 - result: $(D_0 \cup D_+) - D_-$

Index compression

- Index compression needed to fit entire index or its large part to memory...
- Content of index represented with integers
- Basic idea:
 - Integers represented with 4 bytes (32 bits)
 - Many integers smaller than 4 bytes – use **integer coding**.
 - Doc ids can be very large numbers – use **gaps** (id offsets) instead.
 - Example, IDs: 4, 10, 300, and 305. Represented with offset, 4, 6, 290 and 5. Frequent terms have smaller gaps!

Types of integer coding

Coding type	Coding	Rules
Variable-bit scheme	- Unary coding - Elias Gamma coding - Elias Delta coding - Golomb coding	for int x, prefix 1 with $x - 1$ zeroes for int x, prefix 1 with $\lfloor \log_2 x \rfloor$ zeroes and add x in binary without the most significant bit for int x, represent $1 + \lfloor \log_2 x \rfloor$ in Gamma code and add x in binary without the most significant bit. for int x, represent $q = \left\lfloor \frac{x}{b} \right\rfloor + 1$ in unary and add binary representation of $r = x - qb$ using coding tree.
Variable-byte scheme	Variable-byte coding	Use seven bits in each byte to code int x, with the least significant bit set to 1 in the last byte, or to 0 if further bytes follow.

Exercise

- Represent number 5 in variable-bit coding.

Coding	Rules
Unary coding	for $\text{int } x$, prefix 1 with $x - 1$ zeroes
Elias Gamma coding	for $\text{int } x$, prefix 1 with $\lfloor \log_2 x \rfloor$ zeroes and add x in binary without the most significant bit
Elias Delta coding	for $\text{int } x$, represent $1 + \lfloor \log_2 x \rfloor$ in Gamma code and add x in binary without the most significant bit.
Golomb coding	for $\text{int } x$, represent $q = \lfloor \frac{x}{b} \rfloor + 1$ in unary and add binary representation of $r = x - qb$ using coding tree.
Variable-byte coding	Use seven bits in each byte to code $\text{int } x$, with the least significant bit set to 1 in the last byte, or to 0 if further bytes follow.

Unary: **00001**

Elias gamma: $\text{concat}(001, 01) = \mathbf{00101}$

$\lfloor \log_2 5 \rfloor = 2$, $\text{binary}(5) = 101$

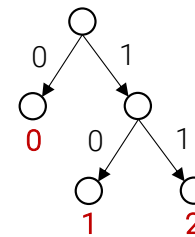
Elias Delta: $\text{concat}(011, 01) = \mathbf{01101}$

$1 + \lfloor \log_2 5 \rfloor = 3$, $\text{gamma}(3) = 011$, $\text{binary}(5) = 101$

Golomb (b=3): $\text{concat}(01, 11) = \mathbf{0111}$

$\text{unary}(\lfloor \frac{5}{3} \rfloor + 1) = 01$, $r = 5 - \lfloor \frac{5}{3} \rfloor 3 = 2$, $i = \lfloor \log_2 3 \rfloor = 1$,

$d = 2^{i+1} - b = 4 - 3 = 1$, $\text{coding_tree}(r) = \text{coding_tree}(2) = 11$



Exercise

- Represent number 135 in variable-byte coding.
-

135 with 4-bytes binary: 00000000 00000000 00000000 10000111

135 with variable-byte coding: 00000011 00001110

Decoding...

■ Elias Gamma

- count zeroes - K
- consider first one after K zeroes as the first digit of the integer, i.e. 2^K
- read the remaining K bits of the integer.

■ Elias Delta

- count zeroes - L
- consider first one after L zeroes as the first digit of the integer, i.e. 2^L
- read the remaining L bits of the integer M .
- use $M - 1$ bits and add one on the first place.

Decoding

■ Golomb

- decode unary-coded quotient q .
- compute $i = \lfloor \log_2 b \rfloor$ and $d = 2^{i+1} - b$.
- retrieve the next i bits and assign it to r .
- if $r \geq d$ then
 - retrieve one more bit and append it to r at the end;
 - $r = r - d$.
- return $x = qb + r$.

■ Variable byte

- read all bytes until a byte with the zero last bit is seen.
- remove the least significant bit from each byte read so far and concatenate the remaining bits.

Exercise

- count zeroes - L
- consider first one after L zeroes as the first digit of the integer, i.e. 2^L
- read the remaining L bits of the integer M .
- use $M - 1$ bits and add one on the first place.

- Decode **00100001** written in **Elias Delta** coding.
-

Number of zeroes: 2

M = first one and remaining 2 bits: $100 = 4$

Result = one plus $M - 1$ leftmost bits: $1001 = 9$

Empirical tests

- **Unary** – nonefficient for large numbers, space consuming
- Parametrized **Golomb** more space efficient and faster for IR than non-parameterized **Elias** coding.
- **Variable-byte** integers often faster than variable-bit integers (fewer CPU operations required for decoding).
- A suitable compression allow retrieval to be up to twice as fast than without compression, while the space requirement averages 20% – 25% of the cost of storing uncompressed integers.

Latent Semantic Indexing

- IR models typically based on **keyword matching**.
- Same concepts can be described with different words (**synonyms**)... important docs might not be retrieved.
- **Latent Semantic Indexing*** (LSI) identifies statistical associations among terms.
- Based on **Singular Value Decomposition** (SVD).

*Deerwester, S., S. Dumais, G. Furnas, T. Landauer, and R. Harshman. *Indexing by latent semantic analysis*. Journal of the American Society for Information Science, 1990, 41(6): p. 391-407.

Formalization

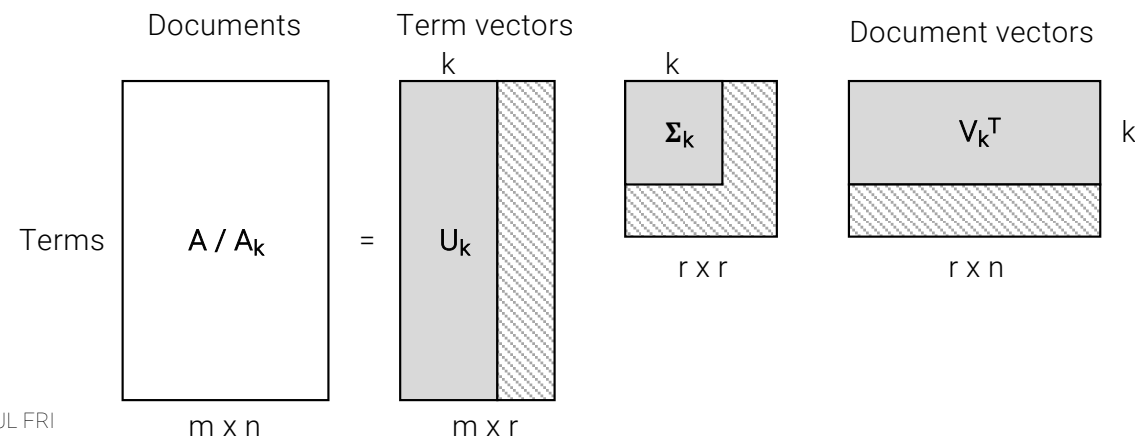
- Let D be the document collection, with m distinctive words and size $|D| = n$.
- Size of term-document matrix A is $m \times n$.
- Let A_{ij} be computed as TF, i.e. the number of times term i occurs in document j .

Singular Value Decomposition

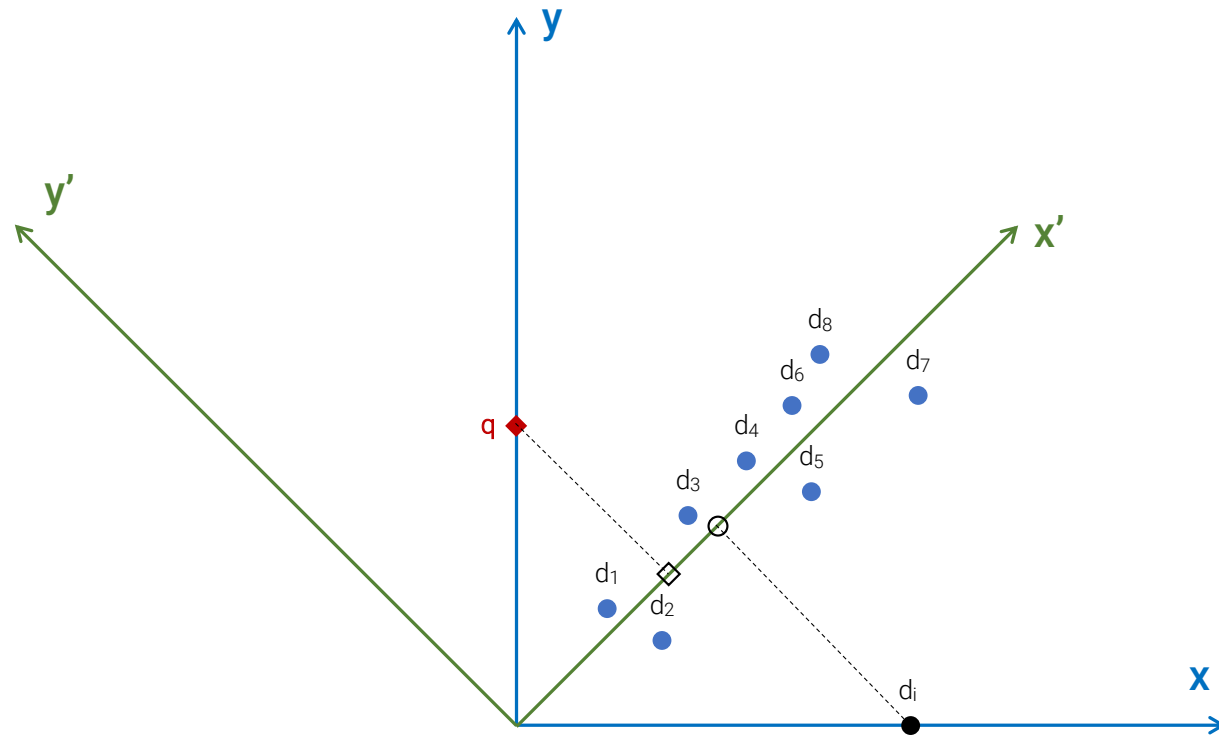
- SVD factors A into the product of 3 matrices: $A = U\Sigma V^T$, where:
 - U is a $m \times r$ matrix with columns that are unit orthogonal vectors called left singular vectors. They are eigenvectors associated with the r non-zero eigenvalues of AA^T .
 - V is a $n \times r$ matrix with columns that are unit orthogonal vectors called right singular vectors. They are eigenvectors associated with the r non-zero eigenvalues of $A^T A$.
 - Σ is a $r \times r$ diagonal matrix, $\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r)$, $\sigma_i > 0$. σ_i called singular values, are the non-negative square roots of the r eigenvalues of AA^T (arranged in decreasing order).
 - r represents rank of A , $r \leq \min(m, n)$

Reducing hidden concept space

- Matrix Σ can be reduced by removing dimensions associated with smallest singular values.
- Let's say we use only k largest singular values in Σ . This leads us to the k -concept space: $A_k = U_k \Sigma_k V_k^T$



Intuition of LSI



Query and retrieval using LSI

- q treated as a document and transformed to k-concept space
- Transformed documents are represented with V_k .
- q_k becomes an additional column/document in V_k^T . Thus q is:

$$q = U_k \Sigma_k q_k^T$$

- and q_k is:

$$q_k = q^T U_k \Sigma_k^{-1}$$

- For retrieval, q_k is compared with other documents in V_k .

Exercise...

- For the below corpus use LSI to find best matching documents for the query *q* = *dies, dagger*.
- Corpus:
 - *d*₁: *Romeo and Juliet.*,
 - *d*₂: *Juliet: O happy dagger!*,
 - *d*₃: *Romeo died by dagger.*,
 - *d*₄: *'Live free or die', that's the New-Hampshire's motto.*,
 - *d*₅: *Did you know, New-Hampshire is in New-England.*

Exercise

■ Procedure:

- pre-process corpus (punctuation/stopwords removal, lemmatization, lowercase...)
- define vocabulary *vocab*
- create TF matrix: A for vocabulary *vocab*
- decompose A using SVD: $A = U\Sigma V^T$
- reduce dimensions of the SVD matrices to the size k : $A_k = U_k \Sigma_k V_k^T$
- calculate query representation in k-concept space: $q_k = q^T U_k \Sigma_k^{-1}$
- for each document d_i as represented in V^T , calculate cosine similarity with q_k .

LSI vs keywords-based methods

- LSI performs better than traditional keywords-based methods.
- Drawbacks:
 - Time complexity of SVD ($O(m^2n)$) too high for large corpuses.
 - Concept space not interpretable.
 - Finding optimal k for dimension reduction hard problem.
- LSI vs word embeddings

From classical IR to web search

- Methods we have covered:
 - Boolean · Vector Space · Probabilistic (BM25) · LSI · Word embeddings
- Designed for **curated document collections** — scientific papers, legal databases, news archives.
- The web violates almost every assumption these models were built on.

Why content alone is not enough

- Query: **jaguar speed**
- Millions of pages mention both terms.
- After BM25 or cosine ranking, the top results differ by tiny margins in similarity score.
- Content similarity answers: "Is this page about the query?"
- On the web, that question has millions of "yes" answers.
- We need a different question: which of these is worth showing first?

What makes the web different from a classical corpus

Classical IR corpus	The web
$10^3 - 10^7$ documents	$\sim 10^{11}$ documents (Google index $\approx$ 400 billion, 2025)
Curated, quality-controlled	Anyone can publish anything
Roughly uniform document type	Tweets, PDFs, forums, product pages, spam farms
No links between documents	Hyperlinks form a graph
No user behavior data	Billions of queries and clicks per day
Static collection	Continuously changing; freshness matters
Trust assumed	Adversarial – SEO, spam, manipulation

More than content: the extra signals

- Signals a modern web ranker combines:
 - Content match — BM25F, proximity, semantic similarity
 - Link structure — who links to this page, and how authoritative they are
 - Anchor text — what other pages say about this page
 - User behavior — clicks, dwell time, query reformulations
 - Freshness — recency, update frequency, trending topics
 - Location & personalization — language, geography, prior history
 - Quality signals — spam classifiers, load time, mobile-friendliness
- Content is the first filter. Everything else reshuffles the results.

Beyond content: page prestige

- The web is not a bag of documents — it is a graph.
- A hyperlink is an editorial act: one author saying "this page is worth reading."
- A page with many incoming links — especially from important pages — is probably important itself.
- PageRank formalizes this intuition into a single, query-independent prestige score per page.
- At query time:
 - $\text{final ranking} \approx \text{content relevance} \times \text{page prestige}$

Evaluation of page reputation

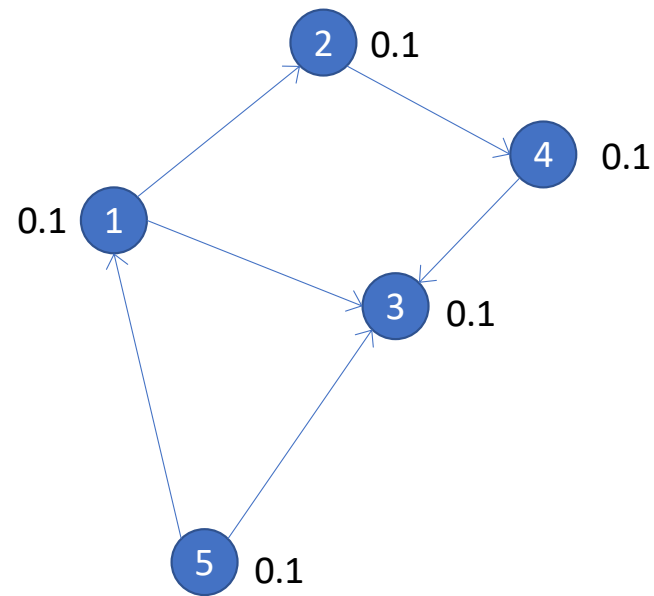
- Page **reputation** computed based on the **link structure** of the page.
- Two algorithms very important: **PageRank** and **HITS**.
- Both introduced in 1998:
 - HITS presented by Jon Kleinberg,
 - PageRank presented by Sergey Brin and Larry Page. Based on the algorithm, they built the search engine Google.

Prestige

- PageRank and HITS based on prestige.
- Prestige types:
 - Degree prestige: an actor is prestigious if it receives many in-links:
 $P_D(i) = \frac{d_I(i)}{n-1}$, where $d_I(i)$ is the in-degree of i and n is the total number of actors in the network.
 - Proximity prestige: only actors that are directly or indirectly connected to i are considered (influence domain): $P_P(i) = \frac{|I_i|/(n-1)}{\sum_{j \in I_i} d(j,i)/|I_i|}$,
where $|I_i|/(n-1)$ is the proportion of actors that can reach actor i .

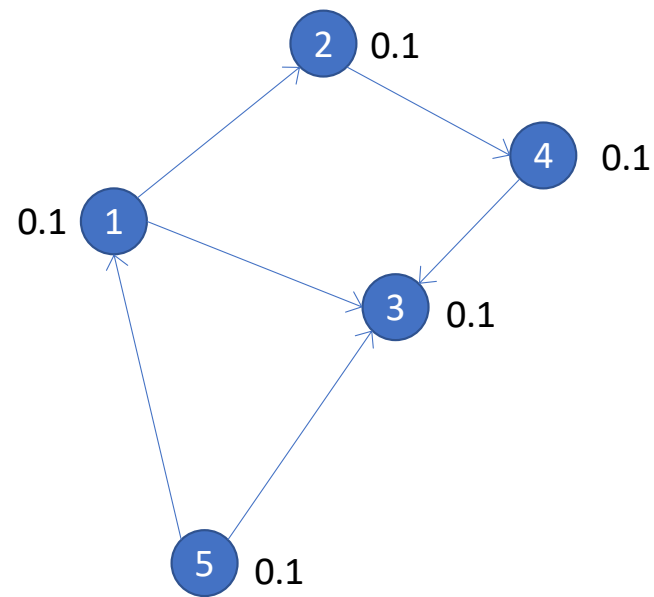
Rank prestige

- **Rank prestige**: considers the prominence of actors that “vote” for the actor under observation.
 - Rank prestige $PR(i)$ is defined as a linear combination of links that point to i : $PR(i) = \sum_j P_r(j)$, where page j points to page i .
 - Let $\mathbf{P}$ represent the vector with rank prestige values of all actors: $\mathbf{P} = (P_R(1), P_R(2), \dots, P_R(n))^T$ and $\mathbf{A}$ matrix where $A_{ij}=1$ if i points to j or 0 otherwise.
 - We then have $\mathbf{P} = \mathbf{A}^T \mathbf{P}$ where $\mathbf{P}$ is eigenvector of $\mathbf{A}^T$ and can be calculated using well known **Power iteration** method.
 - Note: Power iteration not directly useful for PageRank calculation, since $\mathbf{A}$ does not fulfil required preconditions.



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$P = \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$P_1 = A^T * P_0$$

$$P_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.1 \\ 0.0 \end{bmatrix}$$

PageRank

- The basic idea of **PageRank** follows the idea of **rank prestige** with the following difference:
 - Since a page may point to many other pages, its prestige score should be shared among all the pages that it points to. Thus
 - $P(i) = \sum_{(j,i) \in E} \frac{P(j)}{O_j}$, where Web is represented as a directed graph $G = (V, E)$ with V as a set of nodes and E as set of edges or hyperlinks and O_j is the number of out-links of j . Let n be the total number of pages, i.e. $n = |V|$.

$$A_{ij} = \begin{cases} \frac{1}{O_i} & \text{if } (i,j) \in E \\ 0 & \text{otherwise} \end{cases}, P = (P(1), P(2), \dots, P(n))^T, P = A^T P.$$

Modeling Web with Markov chain...

- Markov chain models Web surfing as a **stochastic process** where the surfer randomly and uniformly chooses where to go next.
- We can calculate the probability of a random surfer to be in state j after one step as follows: $p_1(j) = \sum_{i=1}^n A_{ij}(1) p_0(i)$, where $A_{ij}(1)$ is the probability of going from i to j in 1 transition, and $A_{ij}(1) = A_{ij}$. In matrix form: $\mathbf{p}_1 = \mathbf{A}^T \mathbf{p}_0$.
- The probability distribution after k steps is $\mathbf{p}_k = \mathbf{A}^T \mathbf{p}_{k-1}$.

Ergodic Theorem of Markov chains

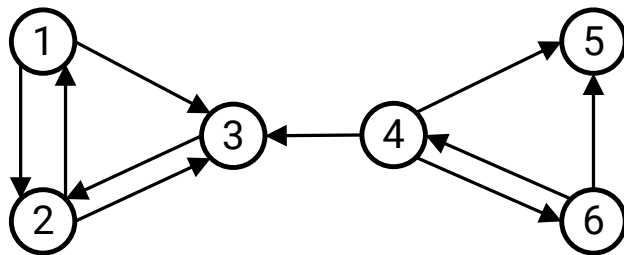
- By the **Ergodic Theorem** of Markov chains, a finite Markov chain defined by the stochastic transition matrix A has a unique **stationary probability distribution** if A is irreducible and aperiodic.
- Stationary probability distribution: after a series of transitions $\mathbf{p}_k$ will converge to a steady-state probability vector $\boldsymbol{\pi}$ regardless of the choice of the initial probability vector $\mathbf{p}_0$, i.e.: $\lim_{k \rightarrow \infty} \mathbf{p}_k = \boldsymbol{\pi}$
- When in steady state, $\mathbf{p}_k = \mathbf{p}_{k+1} = \boldsymbol{\pi}$ which means $\boldsymbol{\pi} = \mathbf{A}^T \boldsymbol{\pi}$ and $\boldsymbol{\pi}$ is the principal eigenvector of $\mathbf{A}^T$ with eigenvalue of 1.

Stationary probability distribution

- Matrix A has to be:
 - **Stochastic**: a stochastic matrix is the transition matrix for a finite Markov chain whose entries in each row are non-negative real numbers and sum to 1.
 - **Irreducible**: irreducible means that the Web graph G is strongly connected.
 - **Aperiodic**: a state i is periodic with period $k > 1$ if k is the smallest number such that all paths leading from state i back to state i have a length that is a multiple of k . If a state is not periodic (i.e., $k = 1$), it is aperiodic. A Markov chain is aperiodic if all states are aperiodic.
- When modeling Web with Markov chain, neither of these requirements is satisfied.

Stochastic matrix

- Example of non stochastic Web subgraph

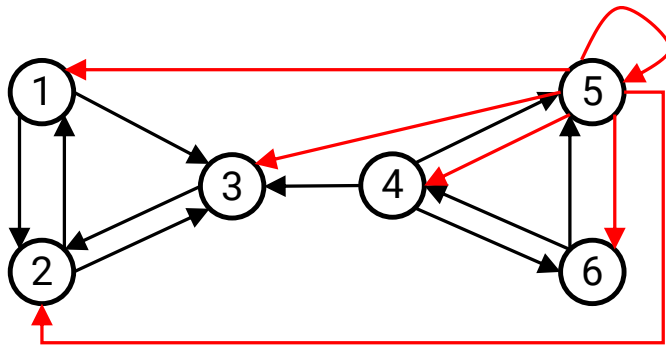


$$A = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 0 & 1/3 & 1/3 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{pmatrix}$$

- Solution:
 - Remove dangling pages including links. Calculate PageRank and then include dangling pages back. Calculate their PageRank.
 - For each dangling page add out links to all pages in the Web.

Irreducible matrix

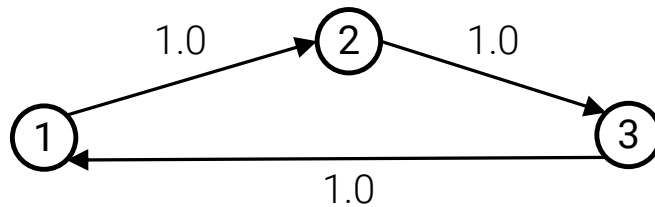
- If we make A stochastic, it can still remain irreducible. There is still no possibility to get from 3 to 4.



$$\bar{A} = \begin{pmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 0 & 1/3 & 1/3 \\ 1/6 & 1/6 & 1/6 & 1/6 & 1/6 & 1/6 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{pmatrix}$$

Aperiodic matrix

- Aperiodic matrix has no periodic states. Web subgraph below is periodic. All states are periodic.



$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Making matrix irreducible and aperiodic

- Solution:

- Add a link from each page to every page and give each link a small transition probability controlled by a parameter d .

- New rules for random surfer:

- with probability d , randomly choose an out-link to follow.
- with probability $1 - d$, jump to a random page without a link.

$$\mathbf{P} = \left((1 - d) \frac{\mathbf{E}}{n} + d\mathbf{A}^T \right) \mathbf{P}$$

- Where $\mathbf{e}$ is vector of all 1 and $\mathbf{E} = \mathbf{e}\mathbf{e}^T$ is a matrix of $n \times n$ of all 1.

PageRank formula

- If we scale P so that $\mathbf{e}^T \mathbf{P} = n$ we get:

$$\mathbf{P} = (1 - d)\mathbf{e} + d\mathbf{A}^T \mathbf{P}$$

- Which for page i is:

$$P(i) = (1 - d) + d \sum_{j=1}^n A_{ji} P(j)$$

$$P(i) = (1 - d) + d \sum_{(j,i) \in E} \frac{P(j)}{o_j}$$

PageRank algorithm

- Algorithm

PageRank-Iterate (G)

$$\mathbf{P}_0 \leftarrow \frac{\mathbf{e}}{n}$$

$$k \leftarrow 1$$

repeat

$$\mathbf{P}_k \leftarrow (1 - d)\mathbf{e} + d\mathbf{A}^T \mathbf{P}_{k-1}$$

$$k \leftarrow k + 1$$

until $\|\mathbf{P}_k - \mathbf{P}_{k-1}\| < \varepsilon$

return $\mathbf{P}_k$

PageRank strengths and weaknesses

- Strengths

- query independent
- hard to cheat

- Weaknesses

- authority in general and not in connection with the query
- time not considered